



BUILD A WORLD MODEL FROM SCRATCH • LECTURE 5

A Vector *or a Vocabulary?*

Our world model has been carrying a handful of numbers. Today we ask a different question: what if it carried *words* instead — and what if it read its own past the way a language model reads a sentence?

RNN vs TRANSFORMER

CONTINUOUS vs DISCRETE

VQ-VAE

IRIS

Vizuara AI Labs · Build a World Model from Scratch

Before anything else – here is what we built

**We trained two world models
to imagine it with the screen turned off.**

600 episodes · 256,151 frames · neither model has seen the level you are about to watch

real CoinRun, then the same scene imagined by two world models with the camera switched off · 600 episodes, 256,151 frames, and a level neither model has ever seen

A dream you can drive, live

Both models watch the same eight real frames. Then the screen goes off — and **you** choose what happens next.

● Press left, right or jump

Each press runs the action through *both* world models and shows you what each one imagines. Nothing is replayed from the game — these frames have never existed.

● See the road not taken

The other two actions are shown beside your choice, so you can watch the futures separate. At step 1 they differ in 2 of 16 tokens; by step 6, in 14 of 16.

● 3,279 real rollouts, entirely offline

Every branch was run on a GPU ahead of time, so it works with no network. By the end of the lecture you will understand every number on that screen.



[simulator/index.html](#) — open it from disk

Four lectures in, one thing has never changed

Every world model we have built carries a **small vector of continuous numbers** between timesteps, and updates it with a recurrent network. Lecture 3's 12 numbers; Lecture 4's 288.

● It worked

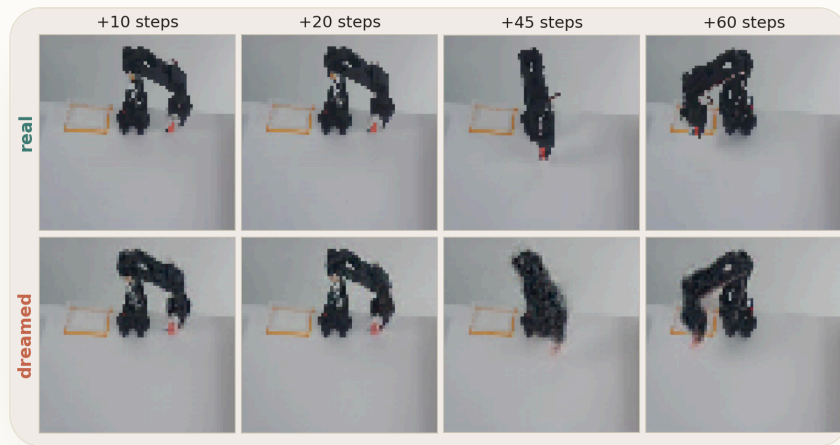
Lecture 4's model dreamed sixty steps of a real robot with the camera off, and a linear probe read the arm's joint angles straight back out of the memory.

● But look closely at the dreams

They are *soft*. The arm is in the right place, the joints are right, and everything is very slightly blurred. We called it a limitation and moved on.

● Today we find out why — and it is not undertraining

It is the **shape of the latent**. And fixing it means changing both things we have never questioned: what the state is made of, and how the model reads its own past.



Lecture 4's imagination. Correct — and soft.

Two ideas, one paper opened all the way up, then a measured comparison

Parts 1 and 2 use **news text**, not pixels — because a vocabulary is something you already understand, and text is where both of today's ideas were proven first.

1 How does a model read a sequence?

A running summary, or a window it can look back through. Measured, on real text.

2 What can a latent variable be?

A point in a continuous space, or a word from a list. Good at opposite things.

3 IRIS: how a picture gets a vocabulary

VQ-VAE, the codebook, and where IRIS sits among the models you know.

4 Inside the transformer, layer by layer

The longest part. One word traced through all six layers of our own trained model.

5 Head to head on CoinRun

The RSSM against the token model — same game, same data, same wall-clock.

Explanation from Rajat

— WHITEBOARD —



PART 1 OF 5

Reading a Sequence

By the end of this part you should be able to say: what a recurrent network actually does to a sequence, what attention does instead, and — with a number, not an opinion — where the recurrent approach breaks.

SUMMARY · BOTTLENECK · ATTENTION

Strip it back: a sequence model does exactly one thing

Take a news article. Cover the next word. Predict it. That is the whole job — and it is *precisely* what our world model does, with frames instead of words.

1 The parallel we will use all lecture

word ↔ frame · sentence ↔ episode · vocabulary ↔ ? — that last one is the question the lecture answers.

2 Why start with text

Because in text you can *read* the model's inputs and outputs. When something goes wrong you can see it with your own eyes, without squinting at pixels.

3 Our corpus

20 Newsgroups — real posts about baseball, hockey, politics, space, medicine and graphics. It ships with scikit-learn, so every figure in the next two parts runs on your laptop with no download.

Idea one: carry a running summary

Read left to right. Keep **one vector**. At every word, update it. That vector is everything the model knows about what came before — and the next slide is a picture of it doing exactly that on a real article.

1

What the update looks like

new summary = $f(\text{old summary}, \text{this word})$. One small network, applied over and over. This is a GRU, and it is the same machine that carried the robot's memory in Lecture 4.

2

Why it is appealing

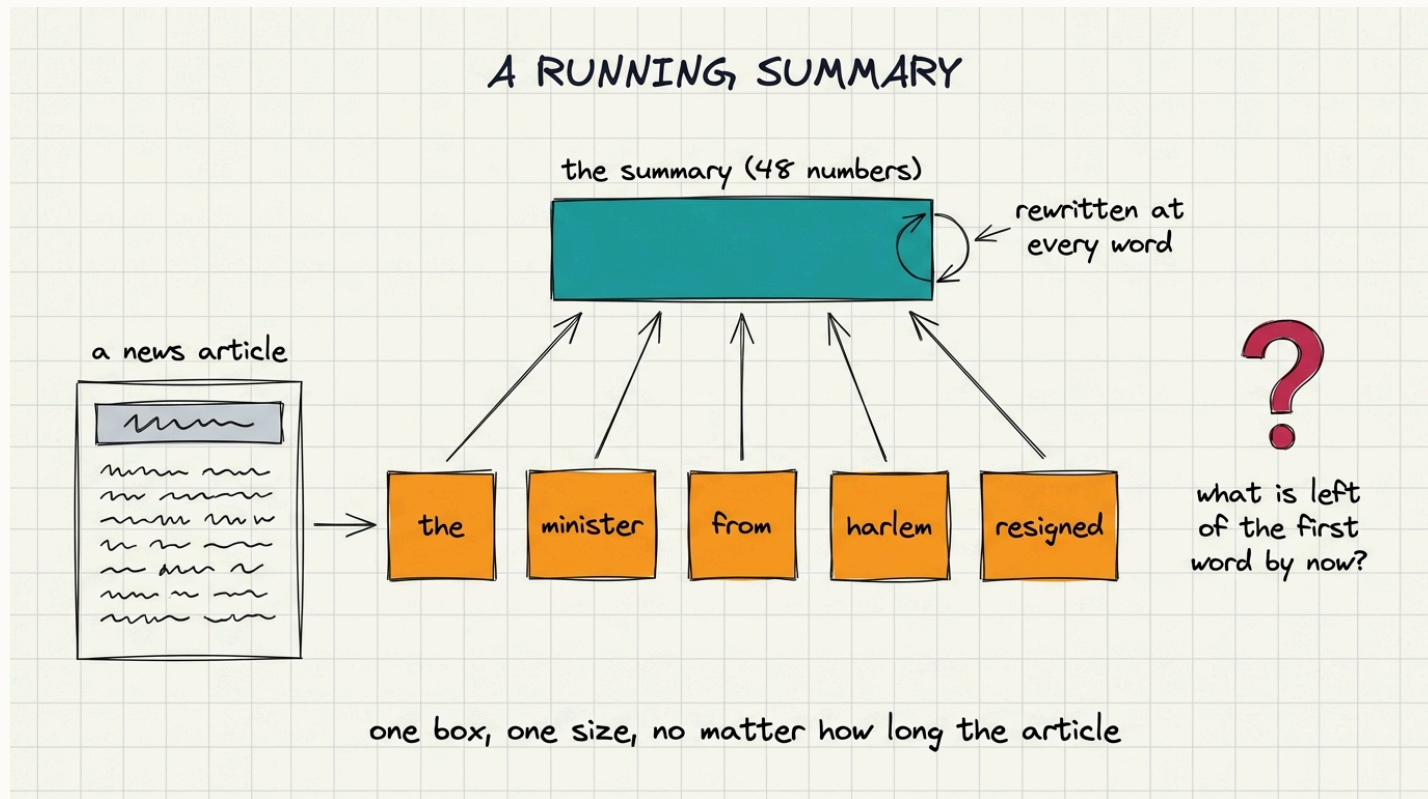
The cost of reading word 1,000 is exactly the cost of reading word 1. Memory usage never grows. You can run it forever.

3

And the catch, stated up front

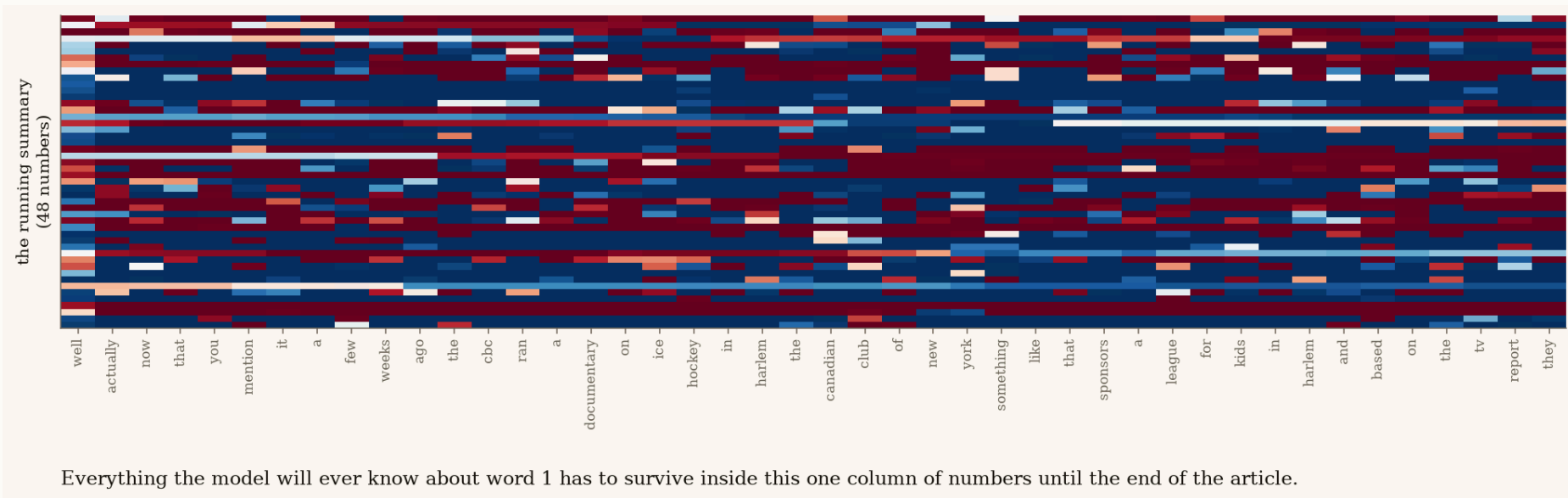
Everything the model will ever know about word 1 has to *survive inside that one vector* until word 400. Nothing else is kept.

One box, one size, no matter how long the article



the summary is the same size after word 5 and after word 500 · everything the model will ever know about the beginning has to survive inside it

An RNN reading a news article – one vector, rewritten at every word



each column is the entire memory after reading that word

Idea two, stated now so the comparison is fair

We are about to race the running summary against something else, so let us say plainly what that something else *is* — the full mechanics come later in the lecture, but you need the one-sentence version first.

1 Keep every word. Do not summarise anything.

The whole passage stays available. There is no box that fills up, because there is no box.

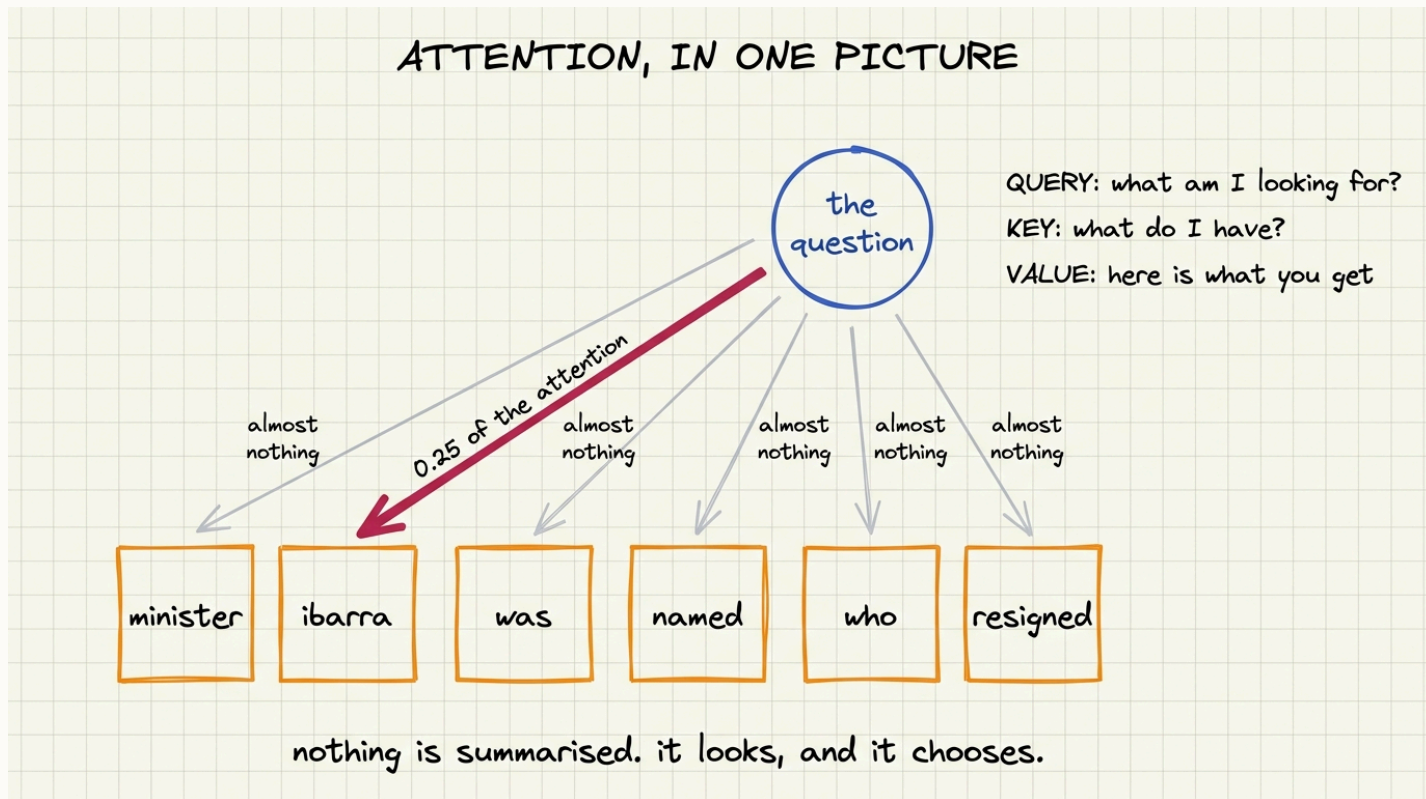
2 When you need something, go and look for it

The position doing the predicting forms a **query** — "I need a person's name". Every earlier word advertises a **key** — "I am a name", "I am a preposition". Strong matches hand over their **value**.

3 That is attention, and that is the whole idea

A weighted average of the past, where the model picks the weights. Distance costs nothing: word 2 and word 200 are equally reachable.

Attention, in one picture



query, key, value — and one thick arrow onto the word that matters · we will open this up properly in Part 4

Don't take our word for it — here is the experiment

The task: a newspaper-shaped passage names a minister in the first sentence, then N words of committee filler, then asks: *who resigned?* The answer is always right there in the text. The only variable is **how far back** it sits.

1

Both models see identical data

Same passages, same training budget, same size. One reads with a running summary; one reads with attention.

2

What we expect

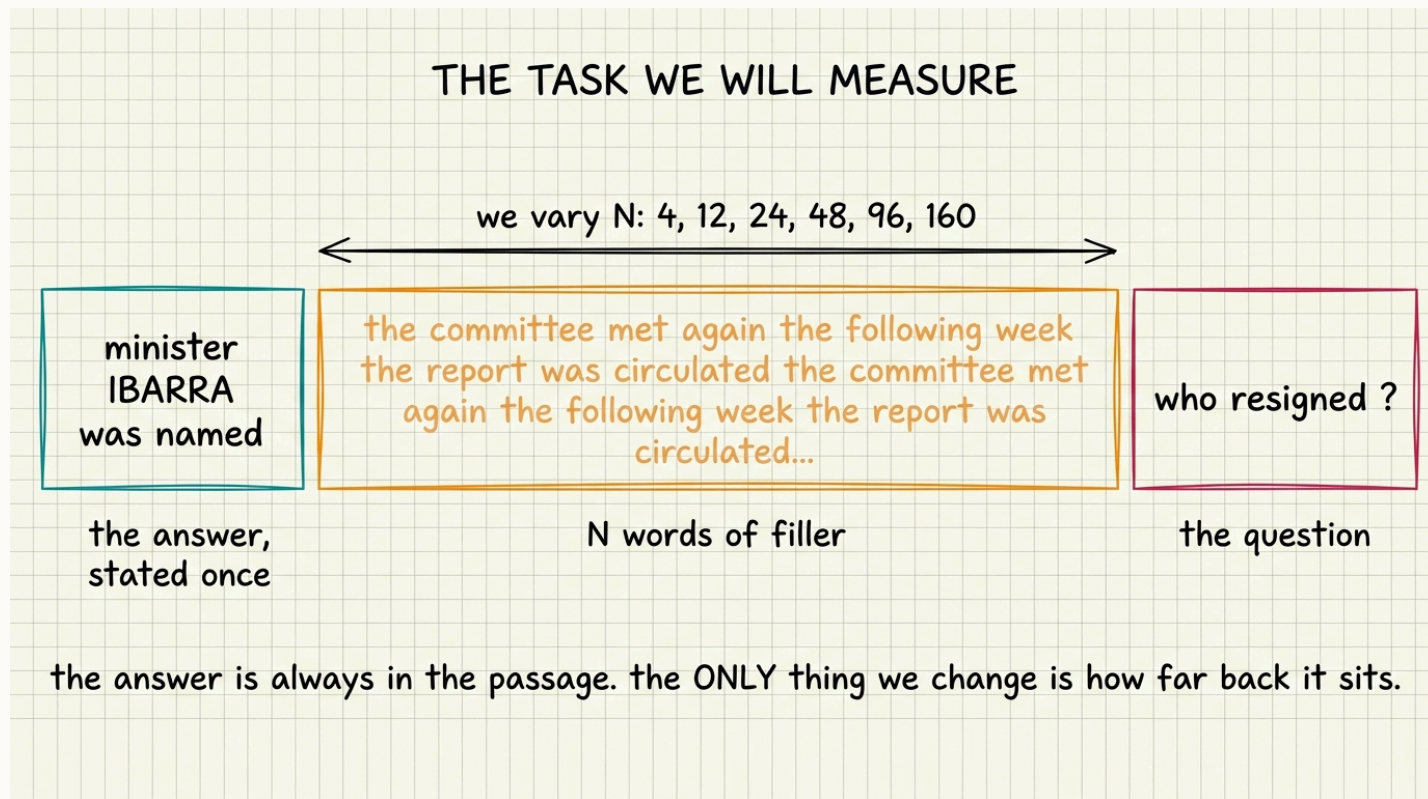
If the summary really is a bottleneck, accuracy should fall as the gap grows — and fall to *chance*, which here is 1 in 8.

3

What actually happened

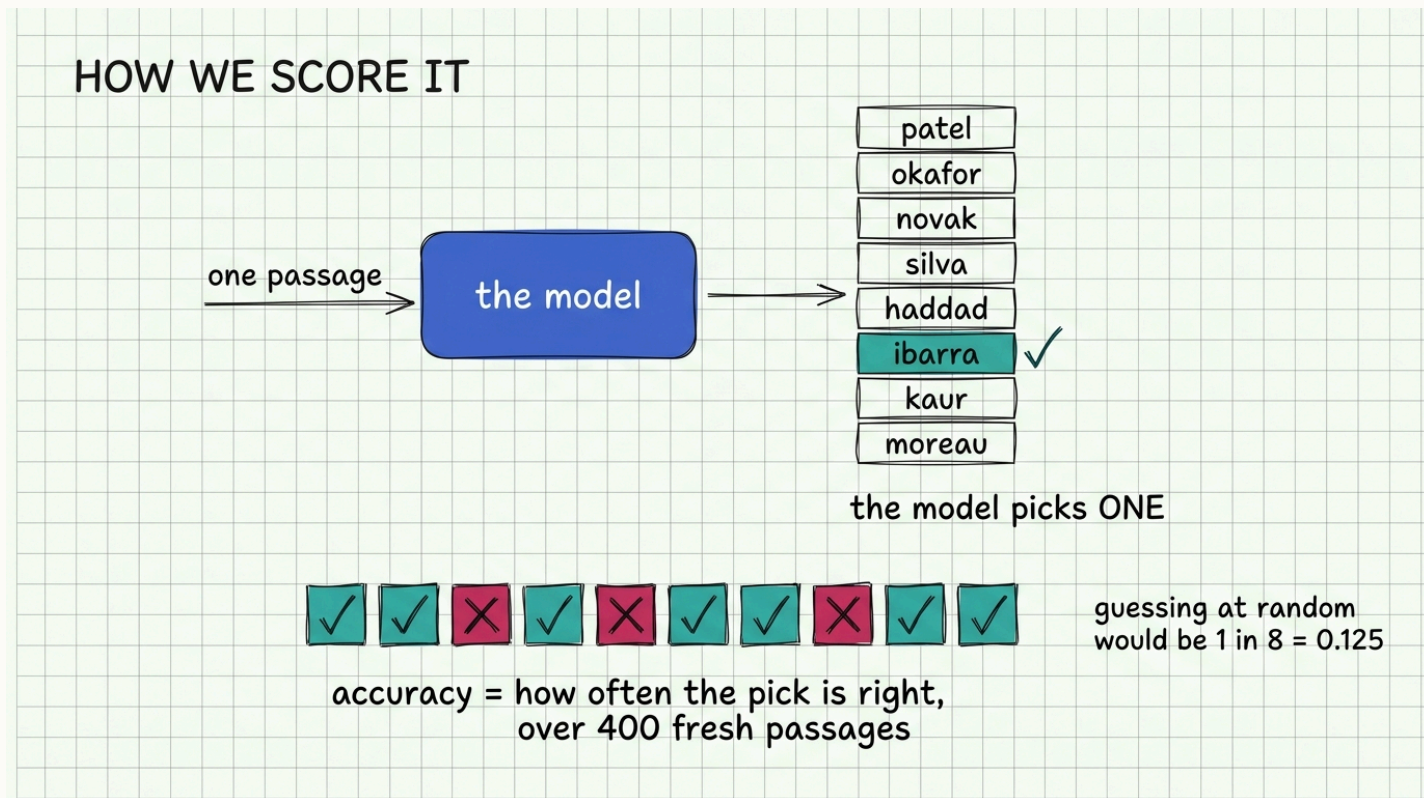
Sharper than we expected. Look at the next slide.

The task, exactly as the models see it



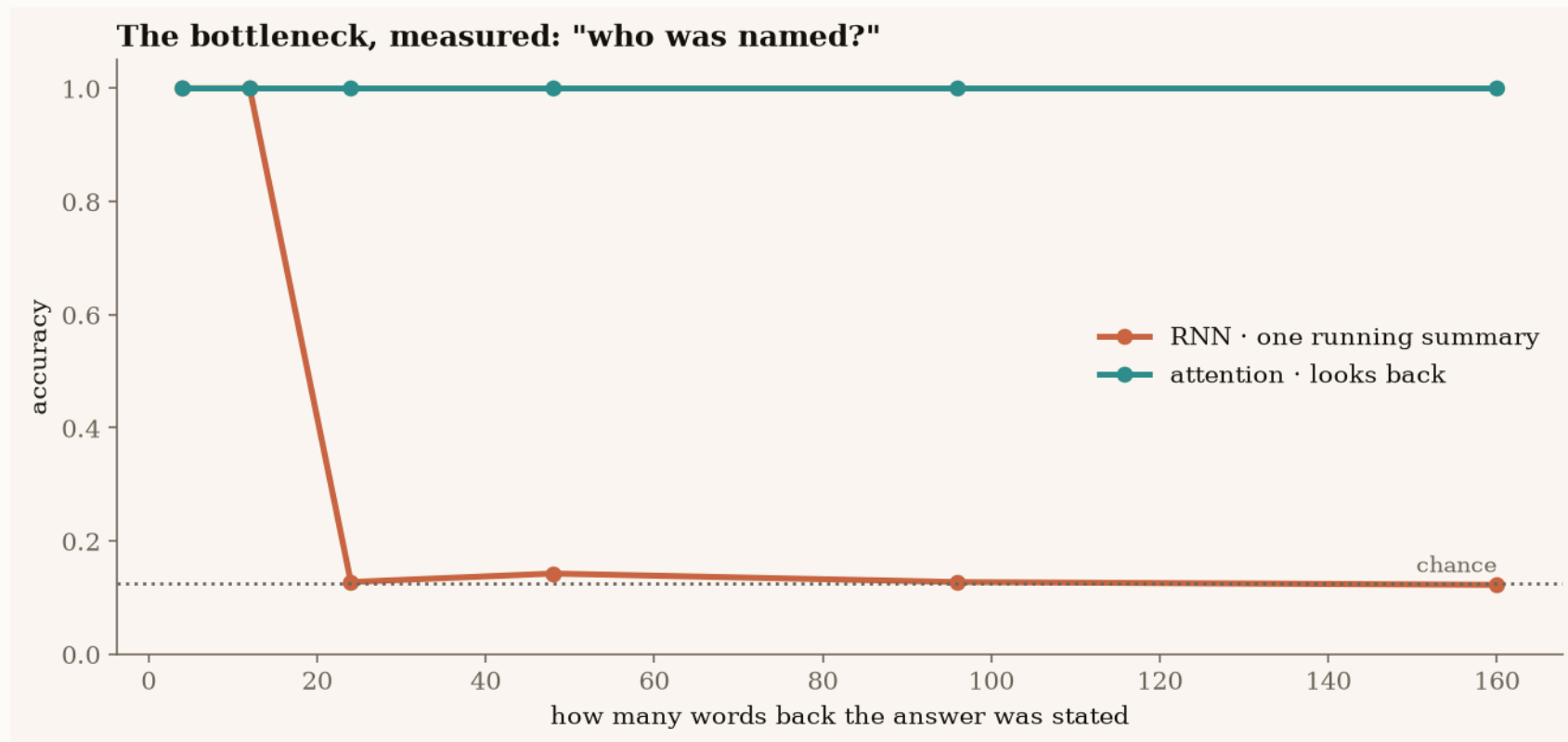
the answer is always present · the only thing we vary is N, how far back it sits — 4, 12, 24, 48, 96, 160 words

And exactly how it is scored



eight possible names, so guessing blind scores 0.125 · accuracy is measured on 400 passages the model was not trained on

The bottleneck is not a metaphor – it is a cliff



the RNN is perfect at a 12-word gap and at **chance** by 24 · attention stays at 1.00 all the way out to 160 words · same data, same budget, same size

A cliff, not a slope — and that is the interesting part

1 Up to 12 words, the summary is fine

The RNN is at 100%. There is nothing wrong with a running summary *as such* — it works, right up until it doesn't.

2 By 24 words it is guessing

0.11 against a chance of 0.125, and it never recovers — 0.14, 0.13, 0.12 at 48, 96, 160. Not degraded: **gone**. The name did not fade gracefully; it was overwritten.

3 And this is exactly Lecture 4's cube

Remember the gripper hiding the cube for fifteen frames? Same failure, same cause: a fixed-size summary competing against everything else that wants to be in it.

Which raises the obvious question: **what if we simply stopped summarising?**

Idea two: keep everything, and look back

Instead of compressing the past into one vector, **keep every word** — and when you need something, go and find it.

1 Every position asks a question

The word being predicted forms a **query**: "I need a person's name."

2 Every earlier position advertises what it has

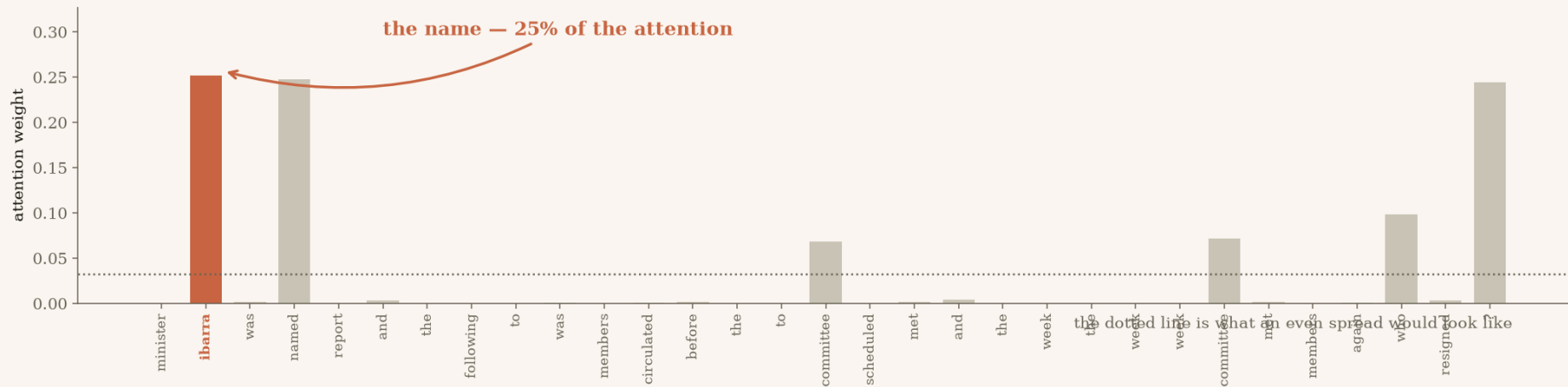
Each past word offers a **key**: "I am a name", "I am a preposition", "I am a date."

3 The matches decide what gets pulled in

Query meets key; the strong matches contribute their **value**. Nothing was compressed, so nothing was lost — the model simply chooses what to look at.

And here is a trained model doing exactly that

Where the question token "?" actually looks — a trained attention model



The name sits 27 words back, past 24 words of filler. The model puts 25% of its attention on it and answers correctly 100% of the time. Nothing was compressed away, so nothing had to be remembered — it simply looked.

the same task the RNN failed · the question token puts a quarter of its attention on a name 27 words back — and answers correctly every time

Three things worth pointing at

1

Nobody told it where to look

We never labelled the name's position. The model discovered that "who resigned?" should attend to the word right after "minister" — purely from being scored on the answer.

2

The 24 filler words get almost nothing

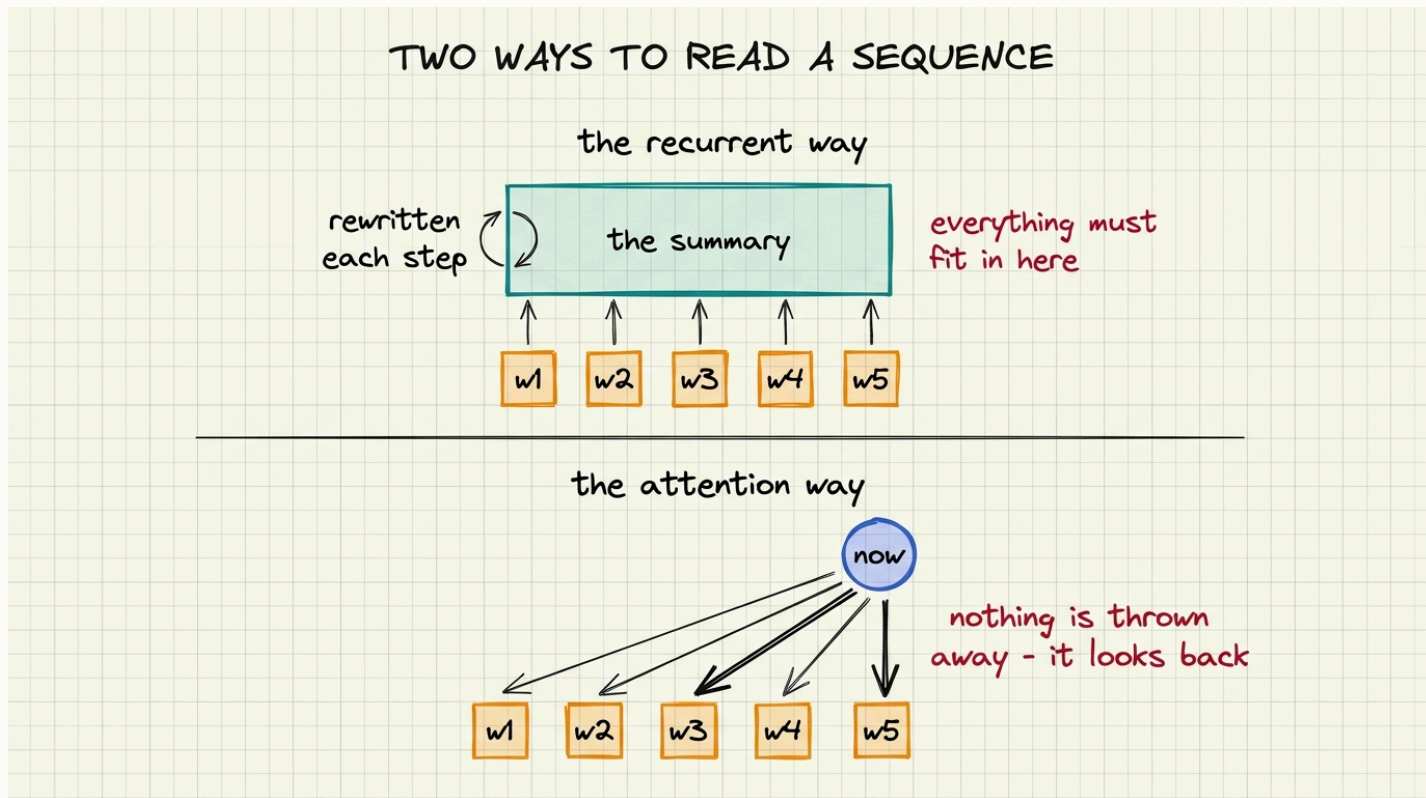
Compare against the dotted line — that is what an even spread would look like. The model is not averaging over the passage; it is *selecting* from it.

3

And the distance simply did not matter

27 words back costs exactly what 2 words back costs. There is no decay, because there is no summary for anything to decay *in*.

Compress and move on, or keep everything and choose



the recurrent way compresses and moves on · the attention way keeps everything and chooses

Three consequences — every one of them returns later

1

Cost per step

Recurrent: constant, forever. Attention: grows with how much you are keeping. *This will decide the last slide of the lecture.*

2

Memory fidelity

Recurrent: lossy, but free. Attention: lossless, but you pay rent on it every single step.

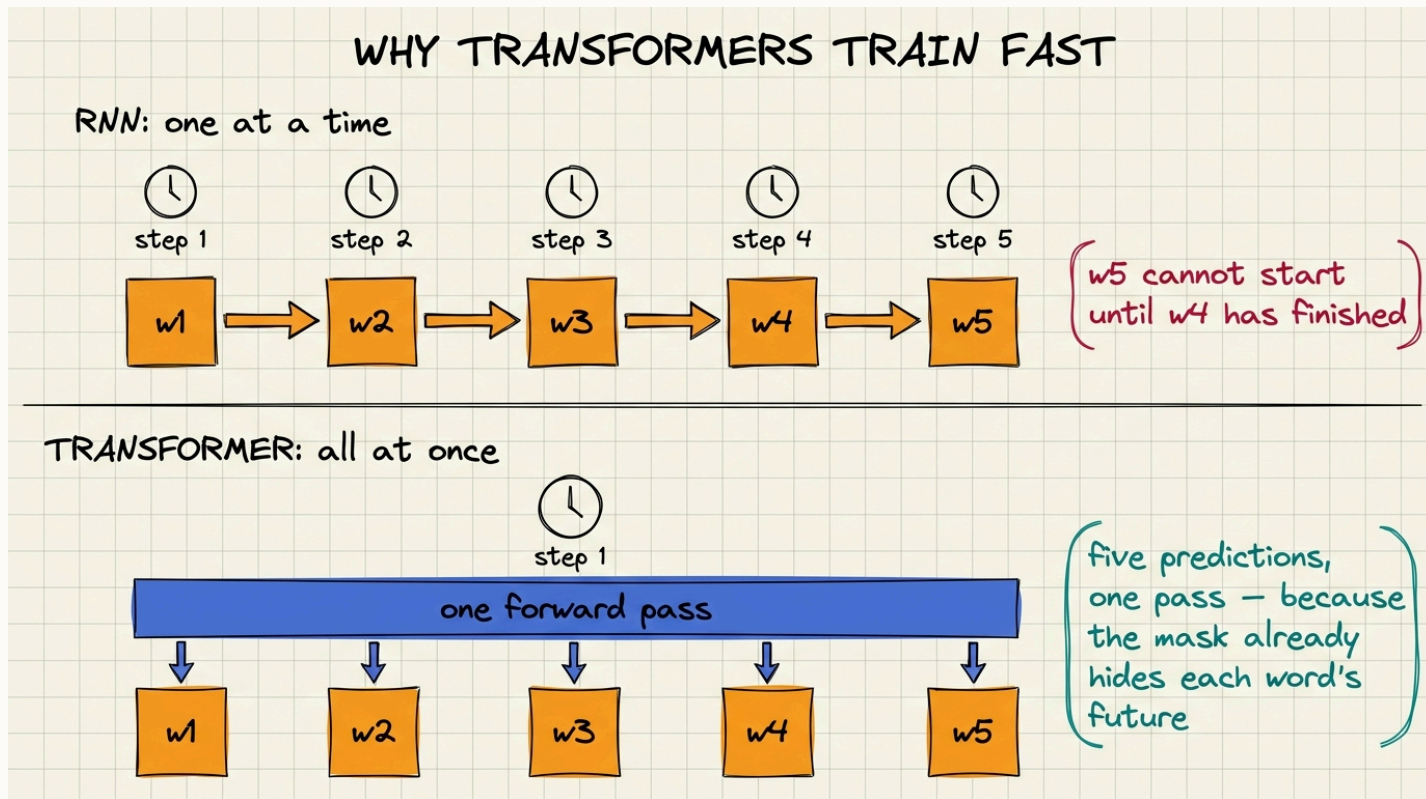
3

Training

Recurrent: strictly sequential — word 5 needs word 4's summary. Attention: every position at once, in parallel. **This, more than anything, is why transformers scaled.**

Notice what we have *not* changed: in both cases the thing being passed around is still a bundle of continuous numbers. That is Part 2.

What "trains in parallel" actually looks like



the RNN must finish word 4 before word 5 can start · the transformer scores every position in a single pass, because the mask already hides each word's own future

Explanation from Rajat

— WHITEBOARD —



PART 2 OF 5

Points or Words

By the end of this part you should be able to say: what a continuous latent buys you, what a discrete one buys you, why they are good at opposite things — and why Lecture 4's dreams came out soft.

INTERPOLATION · ENUMERATION · MULTIMODALITY

What a latent is, and what we have always chosen

A **latent** is a short description of a thing, from which the thing can be rebuilt. Lecture 3: 12 numbers for a Pong frame. Lecture 4: 288 numbers for a robot scene.

1 We have only ever used one kind

Continuous — real numbers, any value, arbitrarily fine. It felt like the only option because pixels are numbers, so codes should be numbers.

2 But there is a second kind

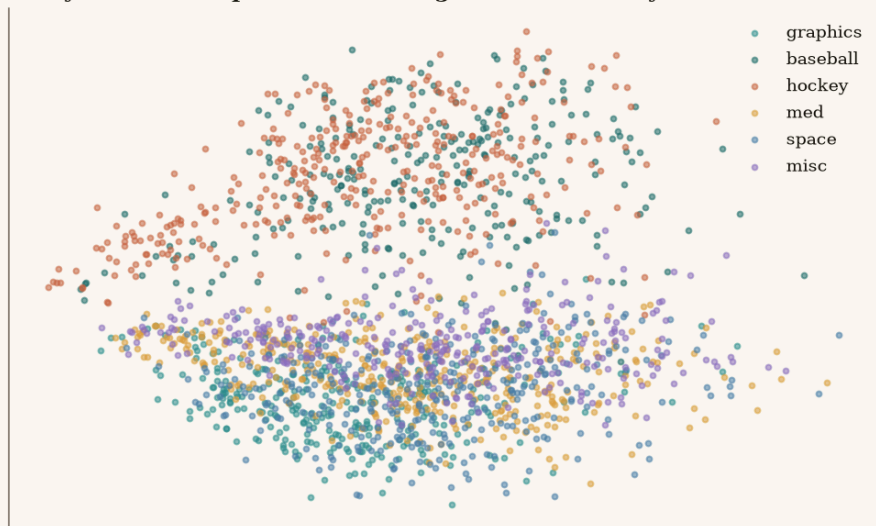
Discrete — a symbol chosen from a finite list. Language works this way: a sentence is a sequence of choices from a vocabulary.

3 Let us look at both, on the same corpus

The same news articles, described two completely different ways — so the difference is about the *representation*, not the data.

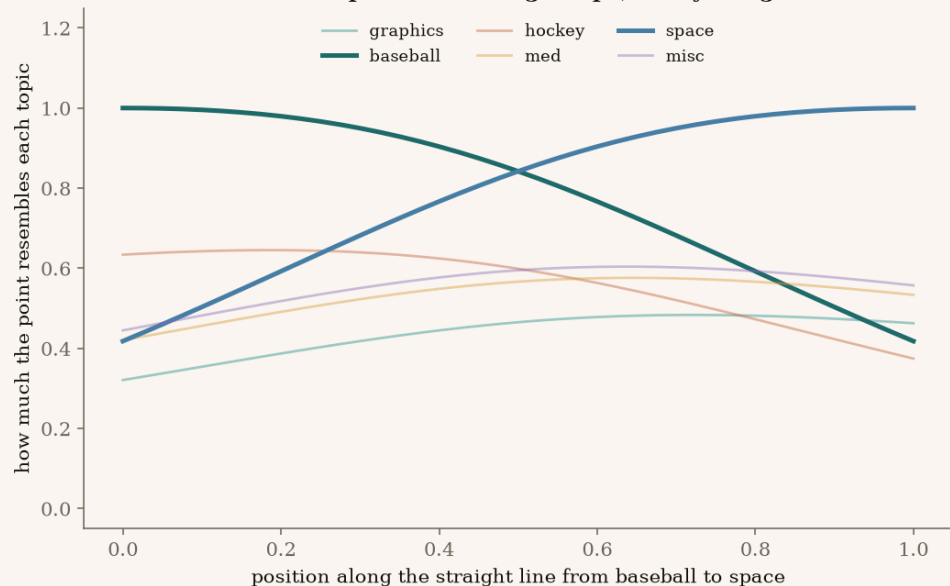
A continuous latent space: every article is a point, and meaning varies smoothly

every article is a point — meaning varies smoothly



*nobody labelled these regions;
closeness means similarity*

walk between two topics — nothing snaps, everything slides



*at 0.5 the point is genuinely half of each —
a real, usable state that is neither article*

2,041 real newsgroup posts across 6 topics, each turned into a 64-number vector (TF-IDF over an 8,000-word vocabulary, then SVD) and projected to 2D · left: topics form regions without ever being told what a topic is · right: walk from the baseball centre to the space centre and every topic score slides

What that map is actually telling you

1

Nearby means similar

Nobody told the model what a topic is. Sports posts landed near sports posts because the space was *built* so that closeness means similarity.

2

The in-between places are real

An article about a hockey players' strike lands between sport and politics — because it genuinely *is* between them. The space has room for that.

3

You can walk from one meaning to another

That is the right-hand panel: step along the line and the nearest real article drifts topic by topic. **A continuous space supports "halfway between".**

This is exactly what our world models have been using — and it is genuinely valuable. Now the other kind.

A discrete latent space: the same corpus, as a vocabulary

the ENTIRE vocabulary is 8,000 words — here are 96 of them, at random

1024x768	114	131	186	1986	226
26	323	995	aas	ace	advanced
asked	author	broken	brunet	comm	converts
coordinates	crap	criminal	damn	development	direction
display	drawing	dynasty	election	evident	faqs
features	fibers	finnish	flew	floating	fractal
girls	glu	hooking	inclination	info	invented
isles	itchy	killer	king	lafontaine	lbl
leaders	limitations	loads	maneuver	mask	migraines
mission	mp	negotiating	new	ninth	october
offensive	ontario	panthers	park	patients	phillips
picking	positive	print	procedure	produced	product
prominent	prompted	regime	regulation	regulations	san
saw	scodal	sinuses	skipjacks	slavery	slugging
sought	spencer	stole	sunday	supportive	teapot
toxins	treatment	unfair	venus	watch	wrigley

a discrete space is enumerable. a continuous one is not.

what is halfway between two words?

baseball
↕
hockey

?

Nothing. There is no word there.
A discrete space does not interpolate —
and that is the price it charges.

left: a discrete space can be **enumerated** — here is the whole thing · right: and there is nothing between two of its entries

So the discrete version looks strictly worse. Why would anyone choose it?

Everything we have shown so far favours continuous: it interpolates, it is compact, it is smooth. Discrete gives that up. There has to be a reason — and there is exactly one, but it is decisive.

1 Ask a question with more than one right answer

"The match was cancelled because of the ____". Rain. Snow. A strike. All plausible, all different.

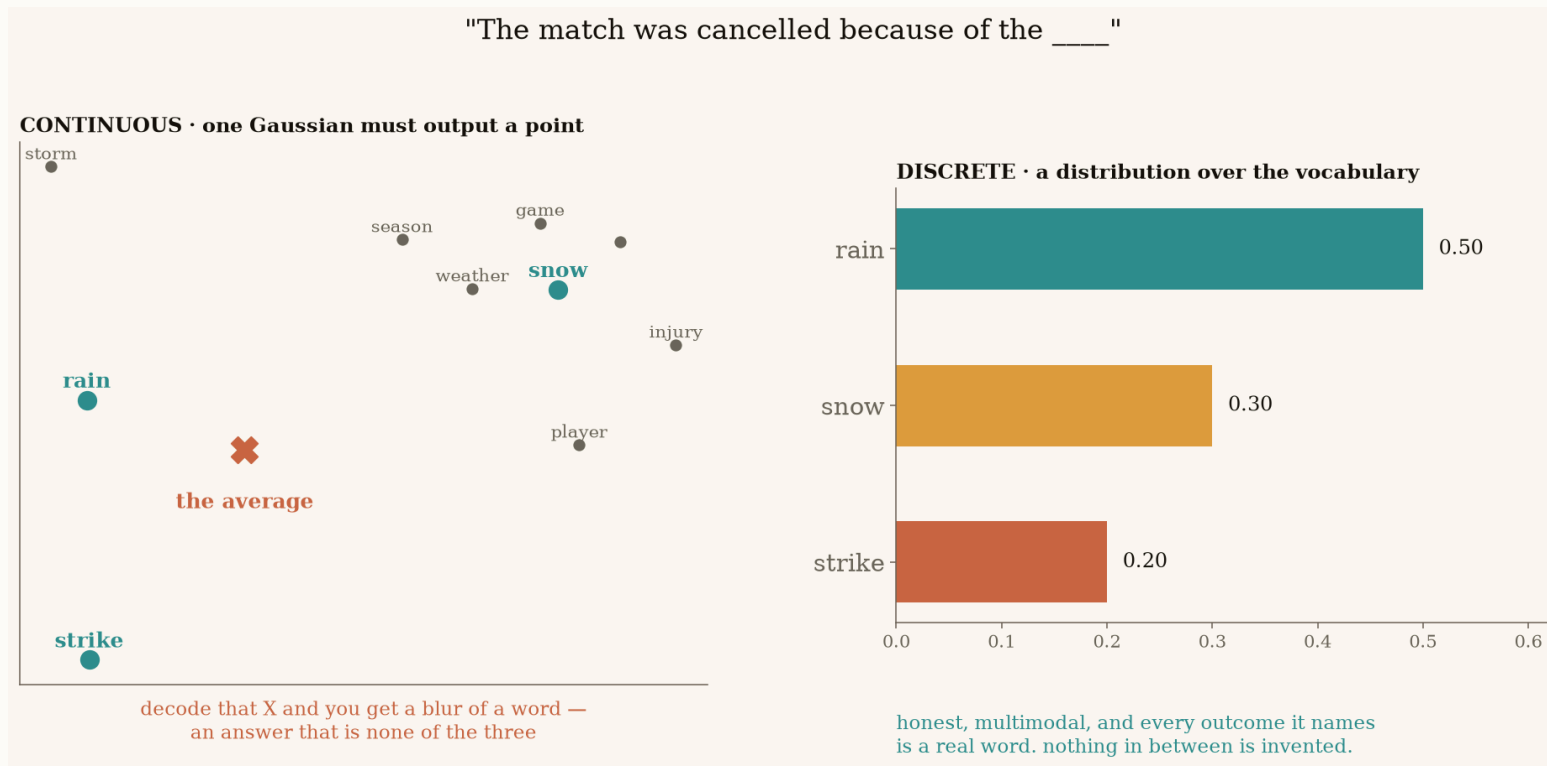
2 A continuous model must answer with a point

One Gaussian has one centre. Asked for a single answer, the safest thing under a squared-error loss is the *average* of the possibilities.

3 And the average of rain, snow and strike is not a word

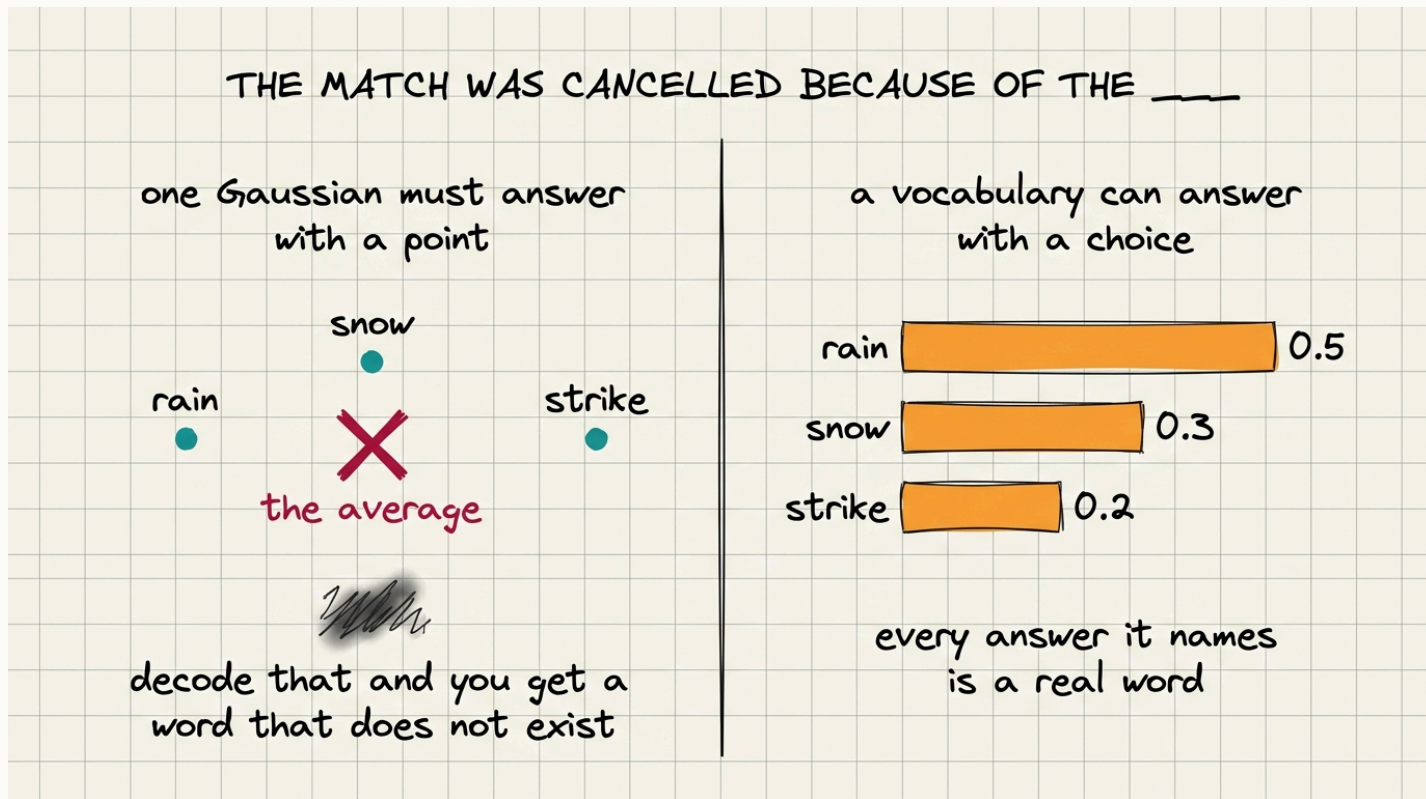
It is a point in the space where no word lives. Decode it and you get a blur. **Watch this next slide carefully — it is the whole lecture.**

The pivotal difference: averaging versus choosing



left: one Gaussian must output a point — and the point between three words is not a word · right: a distribution over a vocabulary names only real options

The same idea, drawn plainly



a distribution over a vocabulary can say "either this or that, and nothing in between" · a single Gaussian structurally cannot

Neither is better. They are good at opposite things.

	CONTINUOUS	DISCRETE
halfway between two states	a real state	nothing exists there
saying "either this or that"	averages them	native
can you list every state?	no	yes — print it
gradients	free	need a trick

1 Continuous wins when the world varies smoothly

A robot arm sweeping through space. A car steering. Positions, angles, velocities.

2 Discrete wins when the world makes choices

Did the grasp catch or slip? Did the platform appear or not? *Either-or* events, where the average is a lie.

"Why not just use several Gaussians?"

The natural first fix, and a real one. Instead of predicting one Gaussian, predict *five* — each with its own centre, width and weight. This is a **mixture density network**, and it is exactly what Ha & Schmidhuber reached for in the original World Models paper. (*We did not build one in Lecture 3 — our MiniPong model used a single Gaussian — so this is new here.*)

1 It genuinely works

Five Gaussians can put mass on rain, on snow and on a strike, with nothing in between. Multimodality, solved.

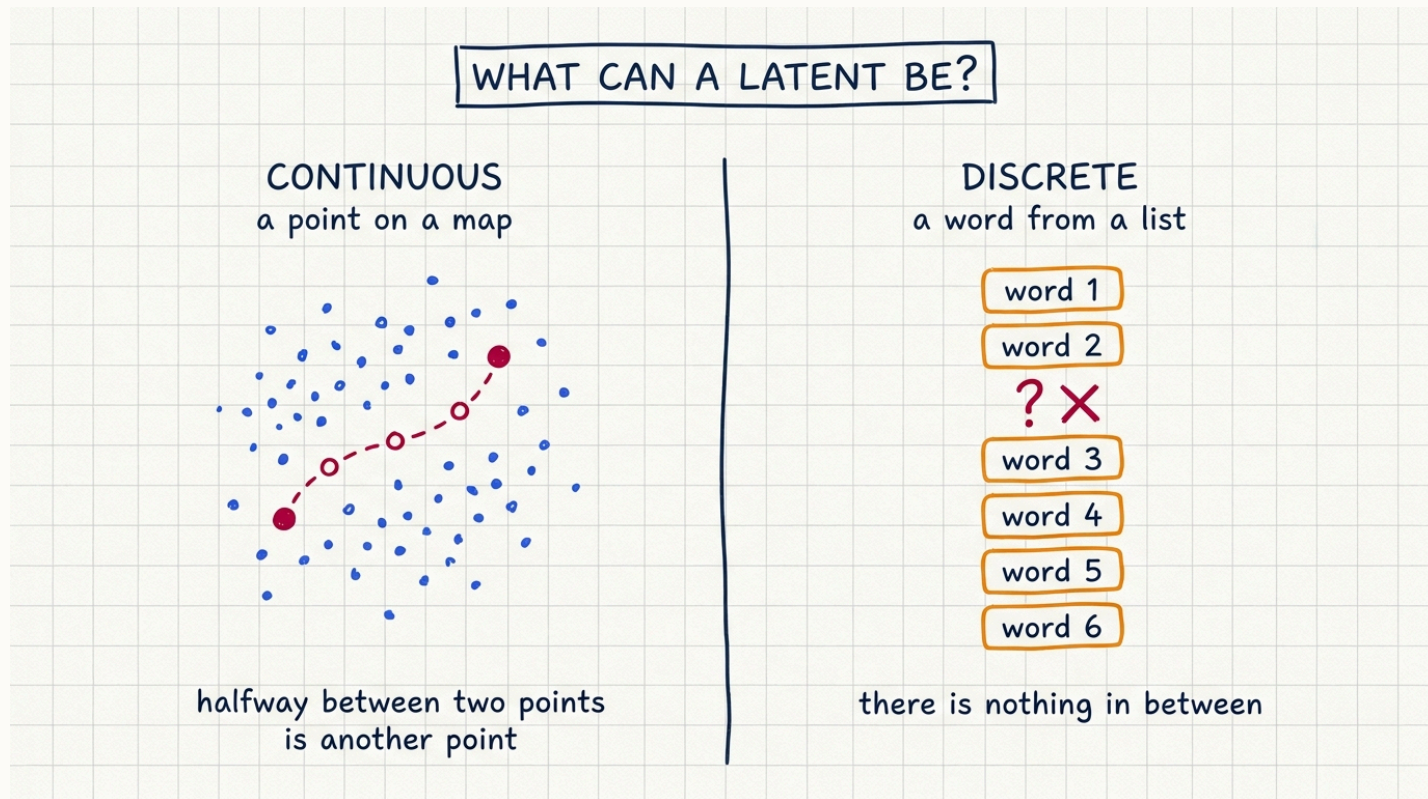
2 But you have to choose the number of modes in advance

Five. Why five? The world does not tell you. Too few and you are averaging again; too many and most of them collapse onto each other.

3 A vocabulary is a mixture with the number chosen for you

512 "modes", one per codebook entry, and the model decides how many to use by how it spreads its probability. It is the same idea taken to its limit — which is why the machinery of language modelling then applies unchanged.

Two kinds of latent, side by side



a point on a map, or a word from a list

Now we can name what happened in Lecture 4

Look again at the dreamed robot frames. They are correct and they are *soft* — and we now have the vocabulary to say exactly why.

● Our latent was one Gaussian

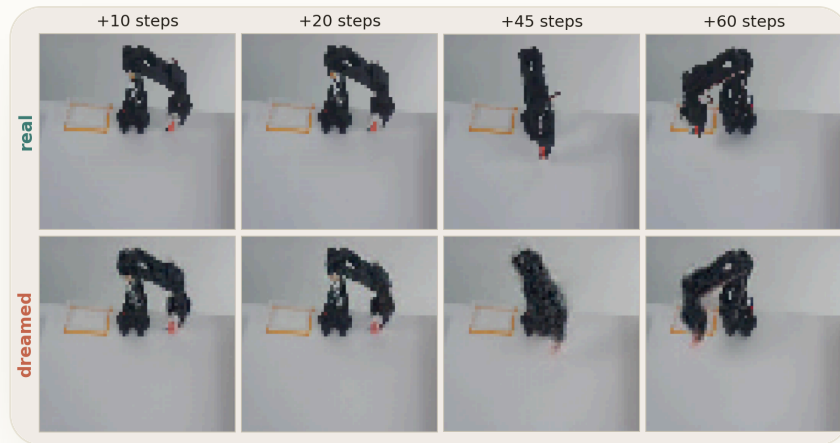
A centre and a width. At any moment where the world could plausibly go two ways, that model has exactly one move available: put the centre between them.

● The decoder faithfully painted the average

An average of sharp frames is a soft frame. The blur is not a training failure — the model was doing precisely what we asked of it.

● So the fix is not more training

It is a different *kind* of latent. Which raises the problem Part 3 has to solve: text arrives with a vocabulary. **A picture does not.**



Lecture 4: the average of the futures it could not choose between.

Explanation from Rajat

— WHITEBOARD —



PART 3 OF 5

Building a Vocabulary

By the end of this part you should be able to say: how a picture gets turned into words when no words exist, what those words actually look like, and how IRIS turns a world model into a language model.

VQ-VAE · CODEBOOK · IRIS

Text comes with a vocabulary. A frame is just 12,288 numbers.

Everything in Part 2 assumed a list of symbols already existed. English hands us one. CoinRun does not — there is no dictionary of "platform", "coin", "sky" anywhere in a screenshot.

1 So we have to invent one

And crucially, *learn* it from the frames themselves — nobody is going to hand-label the parts of a game.

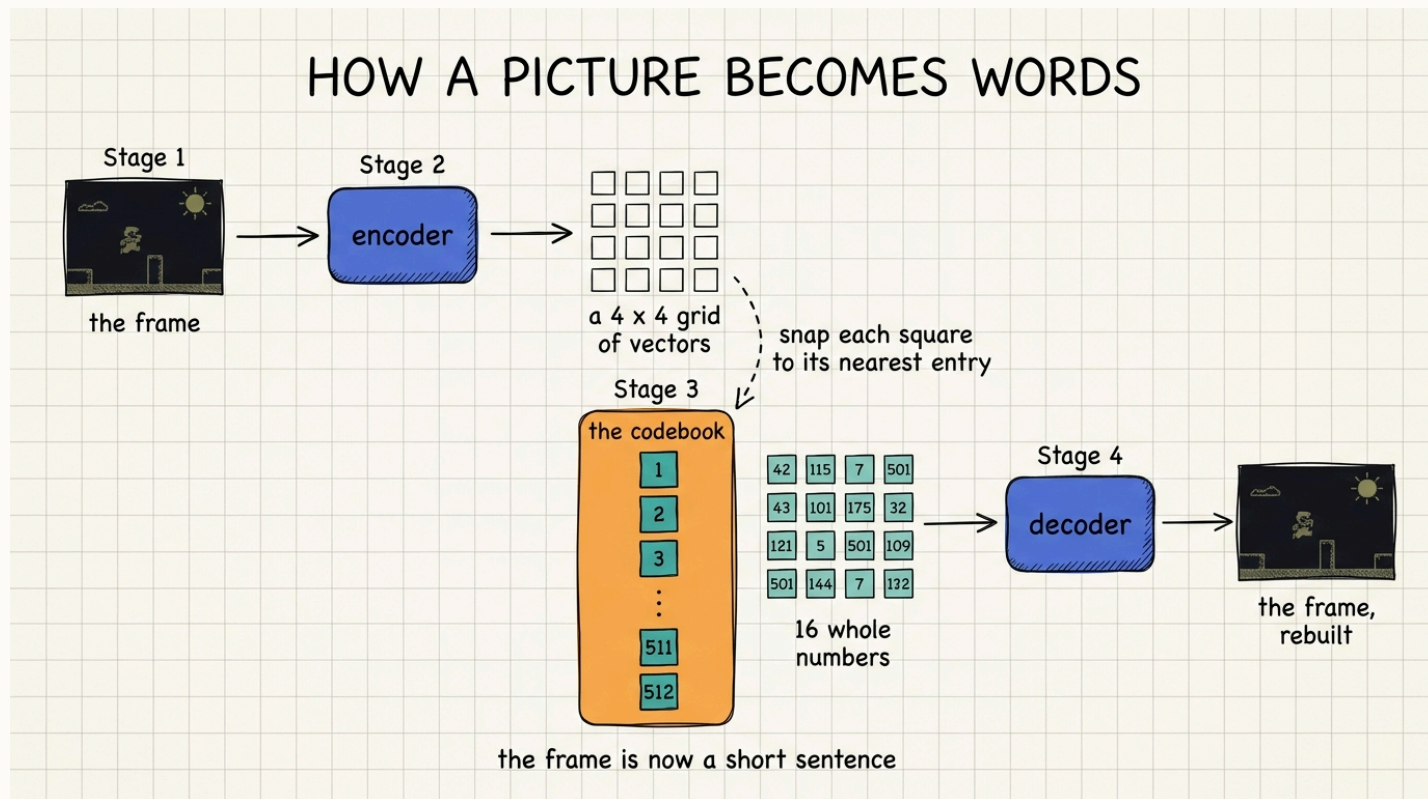
2 The trick: a fixed-size dictionary of vectors

Keep a list of, say, 512 vectors — the **codebook**. To describe a patch of image, find the nearest entry and write down its *number*. The patch is now a word.

3 That is a VQ-VAE

Encoder, snap to the nearest codebook entry, decoder. The only awkward part is that "snap to nearest" has no gradient — and the fix for that is one line.

From 12,288 numbers down to sixteen



a 64×64 CoinRun frame becomes a 4×4 grid of whole numbers — sixteen words drawn from a 512-word dictionary

Sixteen words, not one – and that is what makes it powerful

We could have described a whole frame with a single codebook entry. We deliberately do not. The encoder produces a 4×4 grid, and each cell gets its own word.

1 One word per frame would need a word for every possible frame

512 entries would mean 512 possible scenes in the entire game. CoinRun has vastly more than that. The dictionary would have to be enormous, and it would generalise to nothing.

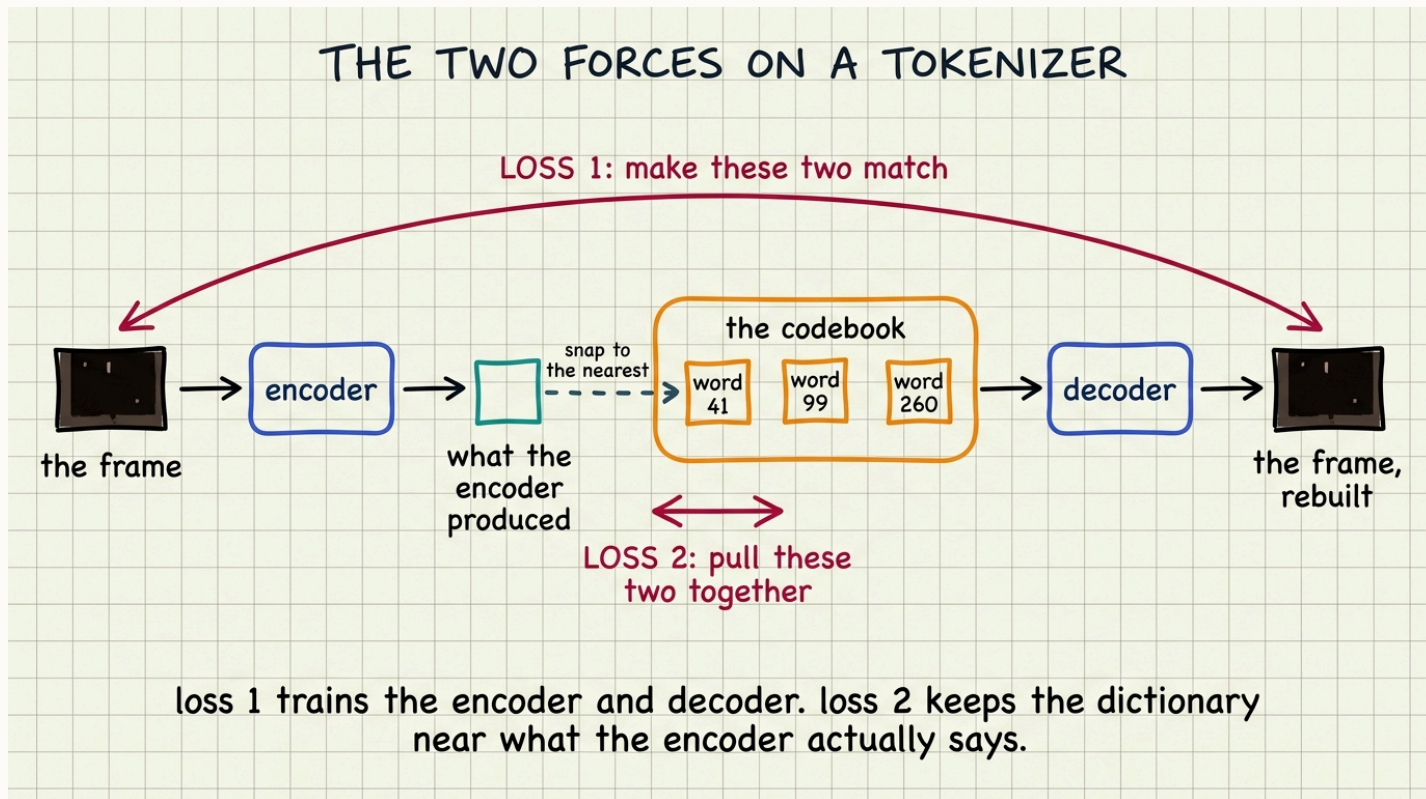
2 With a grid, the words compose

Each cell describes a *region*: this corner is sky, that one is ledge-edge, this one has the player on it. 16 slots × 512 choices is an astronomically large space built from 512 reusable parts.

3 Which is exactly how language works

English does not have a word per sentence. It has ~50,000 words that combine. The frame becomes a short sentence, and now a language model can read it.

The two forces acting on a tokenizer



Loss 1 spans the whole pipeline — the frame must come back · Loss 2 spans one small gap — the word must stay near what the encoder actually said

Two forces, pulling on two different things

A **concrete case**. The encoder looks at a patch of sky and emits some vector. The nearest codebook word is #99, whose vector sits a little way off. We snap to #99 anyway, decode, and get a patch back. *Two things can now go wrong, and there is one loss for each.*

1 Loss 1 • rebuild the frame

Compare the rebuilt patch with the real one, pixel by pixel. If the sky comes back as ground, this is the term that complains. It trains the encoder and the decoder — *choose useful features, and paint them back convincingly*.

2 Loss 2 • keep the words near what the encoder actually produces

Suppose word #99's vector sits far from what the encoder actually produced. Then "snap to nearest" throws away a lot, and the decoder is handed something the encoder never meant. This term shrinks that gap — and it has to pull from *both* ends (move the word toward the encoder, and the encoder toward the word), which is why it appears twice in the paper.

3 How we actually do it

We move the codebook by an **exponential moving average** of the vectors assigned to each entry rather than by gradient — steadier, and standard since the original VQ-VAE. The commitment term stays, keeping the encoder from running away from its own dictionary.

The straight-through trick, and the failure everyone hits

1 "Snap to nearest" has no gradient

Picking the closest entry is a jump, and a jump has a derivative of zero everywhere — so nothing upstream would ever learn. **The fix:** in the forward pass use the snapped vector; in the backward pass pretend the snap never happened and pass the gradient straight through. One line of code, and it works.

2 The classic failure: codebook collapse

Only a handful of entries ever get used; the rest are dead weight. A 512-word dictionary in which the model only ever says three words. We have hit this before, on a previous project — and it is silent, because reconstruction can still look fine.

3 So measure it, always

Track **perplexity** — roughly, how many words the model is really using. We will report ours rather than assume, and there is a fix built in: revive any entry that has gone unused.

IRIS, at the level of a block diagram

Before any mechanism: what are the pieces, what goes in, what comes out. **IRIS is three components**, and the first two are trained separately, one after the other.

1 A tokenizer — trained first, then frozen

A VQ-VAE. *In*: one 64×64 frame. *Out*: 16 integers. It knows nothing about time, actions or the future — it is purely a compressor, and once it is good it never changes again.

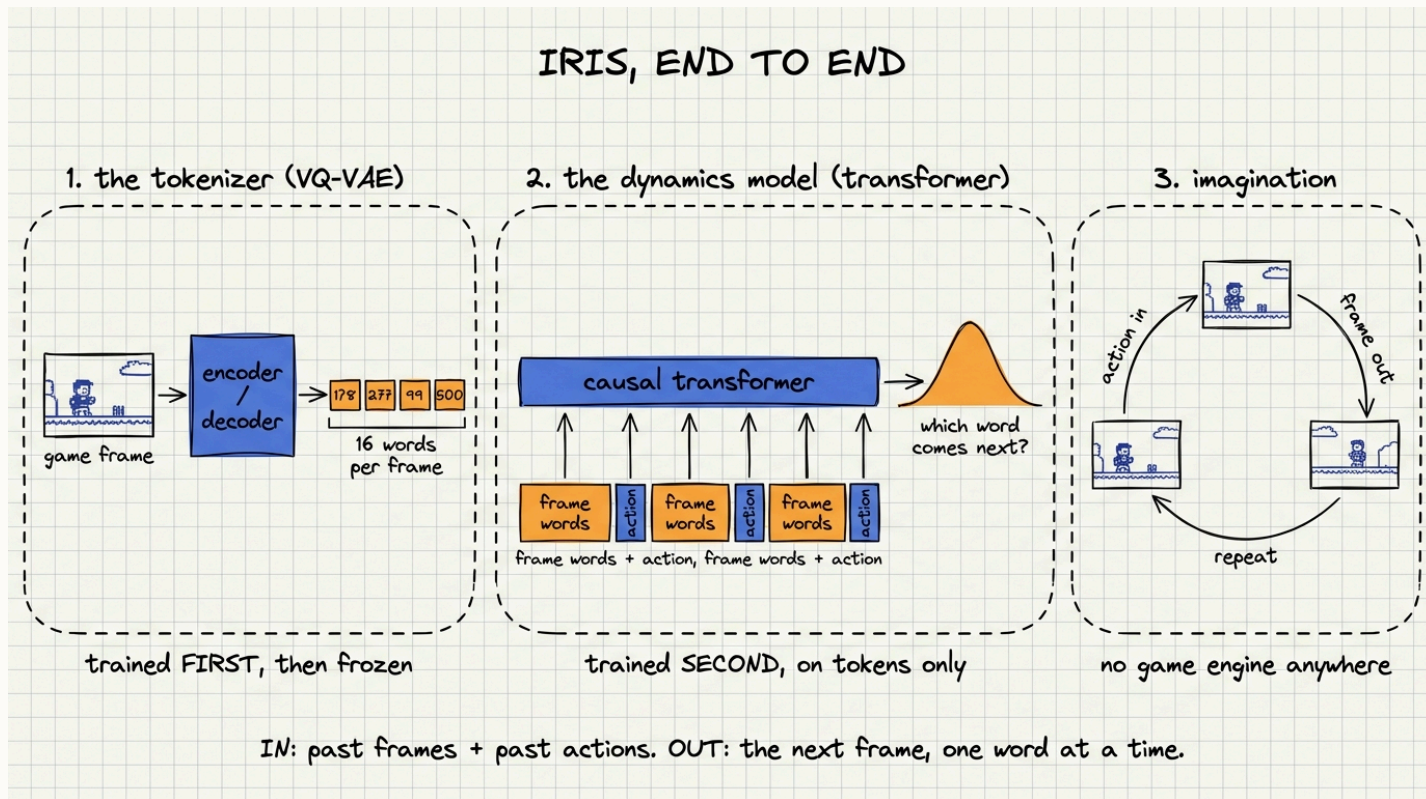
2 A dynamics model — trained second, on tokens only

A causal transformer. *In*: the last few frames as tokens, interleaved with the actions taken. *Out*: a probability over the 512 words for the next token. It never sees a pixel.

3 And in the paper, an agent trained inside the dream

IRIS then trains an actor-critic entirely on imagined rollouts. **We stop before this** — this series is still on the *simulation* half of the taxonomy. Everything we build imagines; nothing yet acts.

IRIS, end to end



IN: past frames and past actions · **OUT:** the next frame, one word at a time · the two trainable stages never run at the same time

The awkward question: why not train it all at once?

1 Because the vocabulary would move under your feet

If the tokenizer kept learning, word #99 would mean something different every few hundred steps. The transformer would be trying to model a language whose words keep changing meaning.

2 Freezing it makes the second stage a pure language-modelling problem

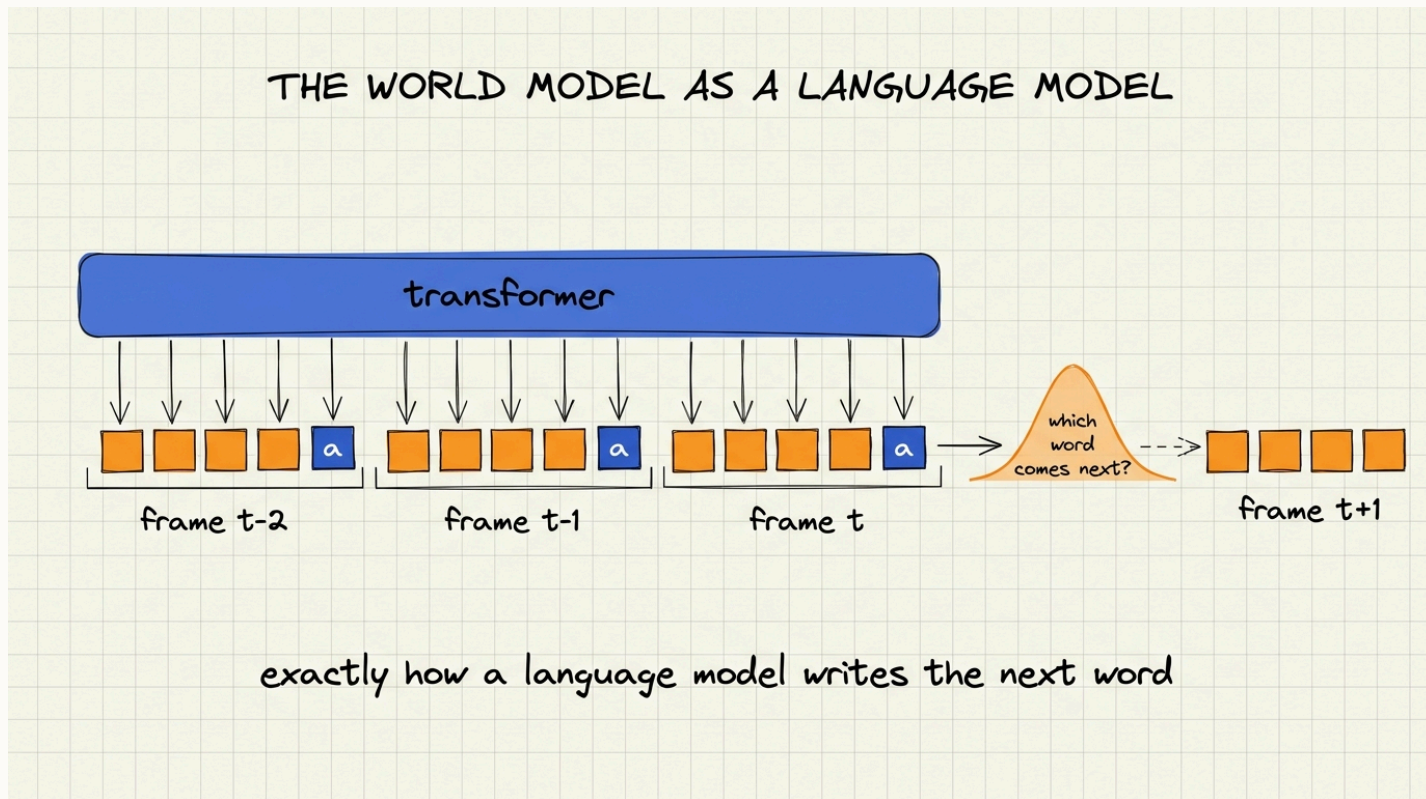
Once the tokenizer is fixed, the whole dataset can be converted to integers *once*. Stage 2 then never touches an image — it trains on a file of numbers, which is why it is so much faster than stage 1.

3 And it is why the pieces are swappable

A better tokenizer drops in without retraining the transformer's architecture, and vice versa. The interface between them is one integer wide.

Exactly what we did: tokenizer for 20,000 steps, then the corpus tokenised once into `coinrun_tokens.npz`, then the transformer for 60,000 steps on that file.

IRIS: the world model becomes a language model



frames become tokens, actions become tokens, and a transformer predicts the next one — exactly how a language model writes the next word

Both of our assumptions, at once

1

The latent became discrete

Sixteen words per frame instead of thirty-two continuous numbers. The model now predicts a *distribution over the vocabulary* — the categorical from Part 2, with everything that buys.

2

The memory became attention

There is no **h**. No belt. The transformer looks back over the token history directly — the fix Part 1 measured.

3

And the whole thing became familiar

Once frames are tokens, a world model *is* a language model. Every trick from language modelling — sampling temperature, top-k, scaling laws — arrives for free.

One honest note on naming: these are two independent choices — *continuous vs discrete* and *recurrent vs attention*. IRIS flips both, which makes it easy to confuse which change did what. Part 4 opens the transformer itself; the experiment in Part 5 keeps the two choices separable.

Two independent choices, and every combination has been built

'FOUR CORNERS, ALL OCCUPIED'

	CONTINUOUS latent	DISCRETE latent
RECURRENT	RSSM PlaNet 2019	DreamerV2 2020
ATTENTION	Dreamer 4 2025	IRIS 2022

the two choices are independent · every combination has been built, and all four are still in use today

Including one you have already met

1

Recurrent + continuous — the RSSM

PlaNet, 2019. Lecture 4. A GRU carrying a Gaussian.

2

Recurrent + discrete — DreamerV2

2020. Same belt, but the Gaussian is replaced by categorical variables. Proof that the two choices really are independent: you can take just the discreteness.

3

Attention + discrete — IRIS

2022. Today's paper.

4

Attention + continuous — Dreamer 4

2025 — and this is **our own CoinRun model**, the one running at worldmodel-platformer.vizuara.ai. A tokenizer with continuous latents and a transformer trained by flow matching. The corner people forget exists.

Explanation from Rajat

— WHITEBOARD —



PART 4 OF 5

Inside the Transformer

By the end of this part you should be able to trace one number end to end: a picture-word goes in as a single integer, and sixteen matrices later it has become a vector that knows about the whole window. We will follow it through every layer of our own trained model.

EMBEDDING · ATTENTION · RESIDUAL STREAM · LOGITS

We have called it "a transformer" for twenty slides. Let's open it.

You have seen that IRIS turns frames into words and predicts the next one. That is the *what*. It is not enough to actually understand the model — so here is the *how*, at full detail, on the model we trained.

1 Everything in this part is traced, not drawn

One real forward pass, on a held-out CoinRun episode. Every heatmap, every number, every attention bar is read straight out of the network.

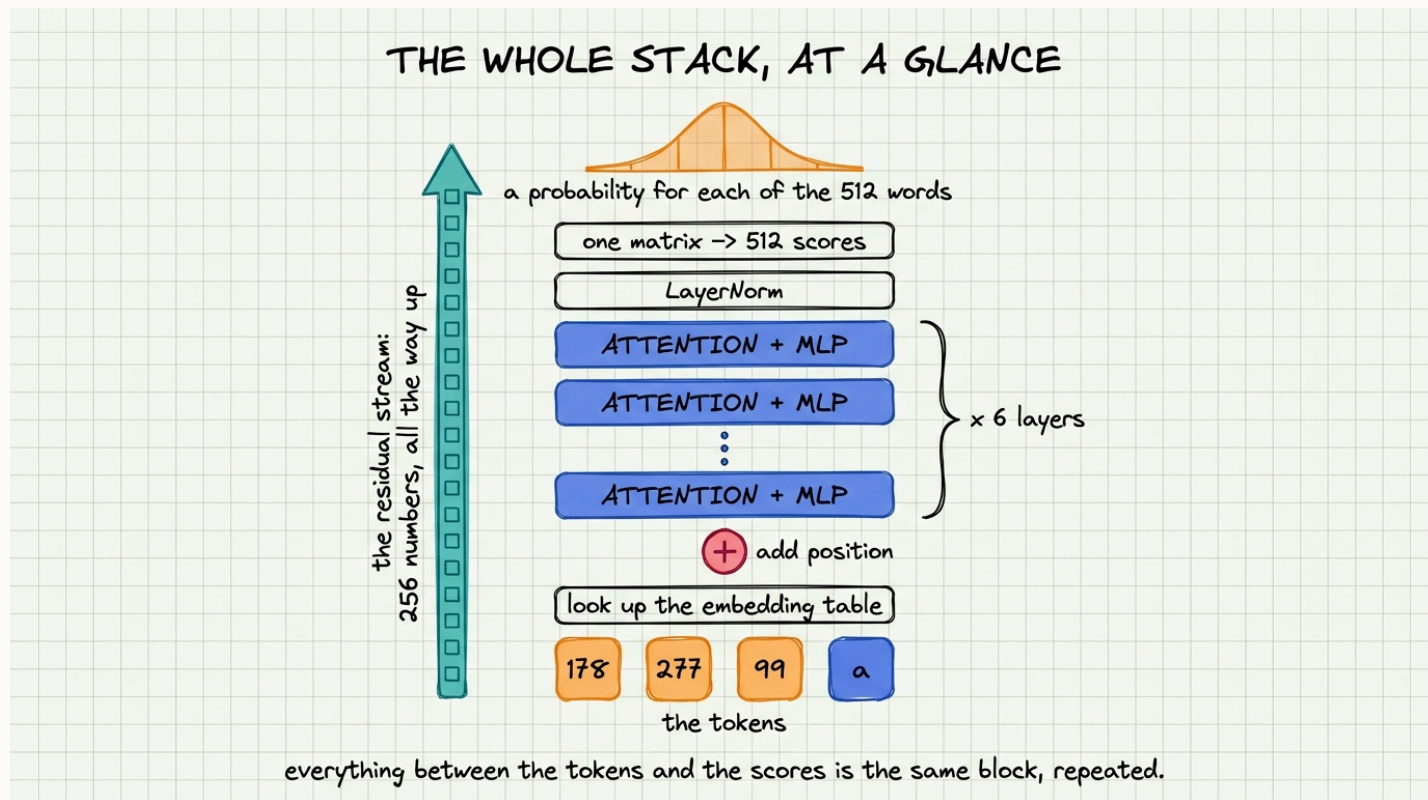
2 The question we are answering

A word enters as the integer **99**. Something leaves the top of the stack that can name the next word. *What happened in between?*

3 The shape of our model

136 positions · **256** numbers per position · **6** layers · **8** heads each · a **512**-word output. Small enough to print, big enough to be real.

The whole stack, before we walk through it



read it bottom to top · everything between the tokens and the scores is *the same block, repeated six times* — the next nine slides walk up this diagram one step at a time

Four things to hold on to before the details

1

Every position keeps its own 256 numbers, the whole way up

136 positions in, 136 positions out. No pooling, no bottleneck. The stack is 136 parallel lanes that occasionally exchange information.

2

Only one operation ever moves information between lanes

Attention. Everything else — the MLP, the normalisations — works on one position at a time and cannot see its neighbours.

3

The block is identical every time

Six layers means the *same shape* six times over, with different weights. Learn one block and you have learned the network.

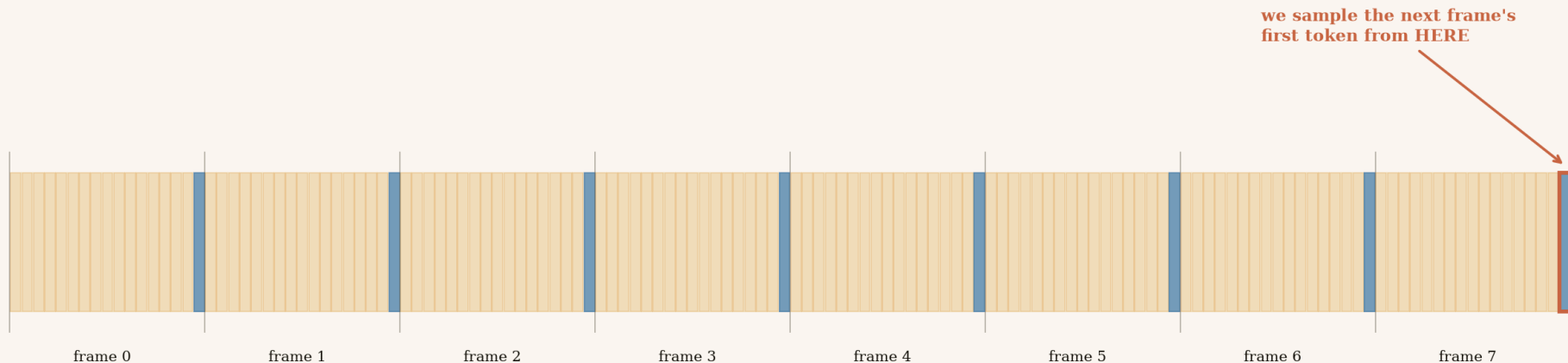
4

Only the last lane is asked a question

All 136 produce an output. When we imagine, we read exactly one of them — the last — and ignore the rest.

Step 1 – what the transformer is actually handed

8 frames × (16 picture words + 1 action) = 136 positions



gold = a word from the 512-word picture vocabulary blue = the action that was taken

eight frames × (sixteen picture words + one action) = 136 positions · this flat stream is the *only* thing the model sees — there is no image and no belt anywhere in it

Three decisions are already baked into that picture

1 A frame is not one thing — it is sixteen

Each frame contributes sixteen separate positions, in a fixed raster order. The model can attend to *part* of a past frame, which a single pooled frame embedding would never allow.

2 The action is a token like any other

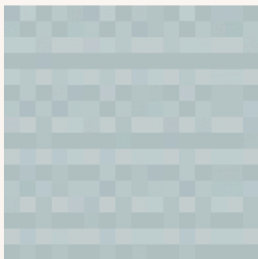
Not a side input, not a conditioning vector — a seventeenth position with its own embedding table. This is why the same architecture works unchanged for text, video and control.

3 The position we care about is the last action token

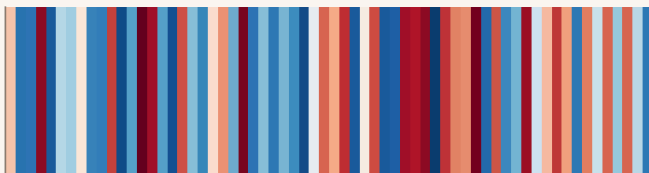
Its output predicts the **first word of the next frame**. Everything in this part traces that one position. Get its loss mask wrong and the model can finish a frame but never start one — which is a mistake we actually made, and it is in the notes.

Step 2 – the integer becomes a vector (and it is NOT the codebook vector)

word #99

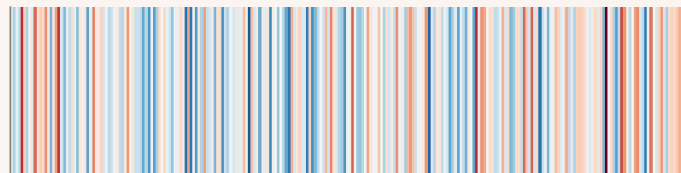


the CODEBOOK vector — 64 numbers



lives in the tokenizer. Used to REBUILD the picture.
The transformer never sees it.

the EMBEDDING vector — 256 numbers



a SEPARATE table the transformer learned for itself.
This is what actually enters the stack.

what it paints

only the integer 99 passes between them · cosine similarity between the two is undefined — they do not even live in the same space

word #99 shown as what it actually paints in the data · the tokenizer's codebook vector and the transformer's embedding are two different objects for the same word, joined only by the integer

The codebook vector never enters the transformer

This trips almost everyone, so it is worth being blunt about. It is natural to picture the 64-number code sliding out of the VQ-VAE and into the attention stack. **That is not what happens.**

1 The tokenizer's job ends at the integer

Encode → snap to nearest → emit **99**. The 64-number codebook vector exists so the *decoder* can repaint the patch. That is its only job.

2 The transformer keeps its own dictionary

A separate 512×256 embedding table, learned from scratch during dynamics training. Word 99 looks up *its* row.

3 And that is exactly what a language model does

The characters "c-a-t" are not a vector; `cat` is token 5372, and GPT looks up row 5372 of its own table. The two halves of IRIS are joined by an *integer*, which is precisely why the tokenizer can be frozen and the transformer trained on top of it.

An embedding table is just a matrix, and a lookup is just a row

"Look up the embedding" sounds like a database query. It is simpler and more mechanical than that, and being precise here makes the next few slides much easier.

1 The table is a 512×256 matrix of ordinary weights

One row per word in the vocabulary, 256 numbers wide. **131,072 parameters**, trained by gradient descent exactly like every other weight in the network. Nothing special about it.

2 The lookup selects row 99 and copies it out

That is the entire operation. (Formally it is a one-hot vector times the matrix — which is why gradients flow back into exactly one row per occurrence.)

3 So the model chooses what its own words mean

At the start these rows are random — word 99 means nothing. They only acquire meaning because the loss pushes words that behave alike toward similar rows. *The vocabulary was given by the tokenizer; the meanings are the transformer's own.*

Attention has no idea what order anything is in

Worth pausing on, because it is genuinely strange the first time you meet it. Attention computes a weighted sum. A sum does not care about order — shuffle the input positions and you get the same answer.

1 Which would be fatal here

"Player left, then right" and "player right, then left" would be indistinguishable. A world model that cannot tell those apart is not a world model.

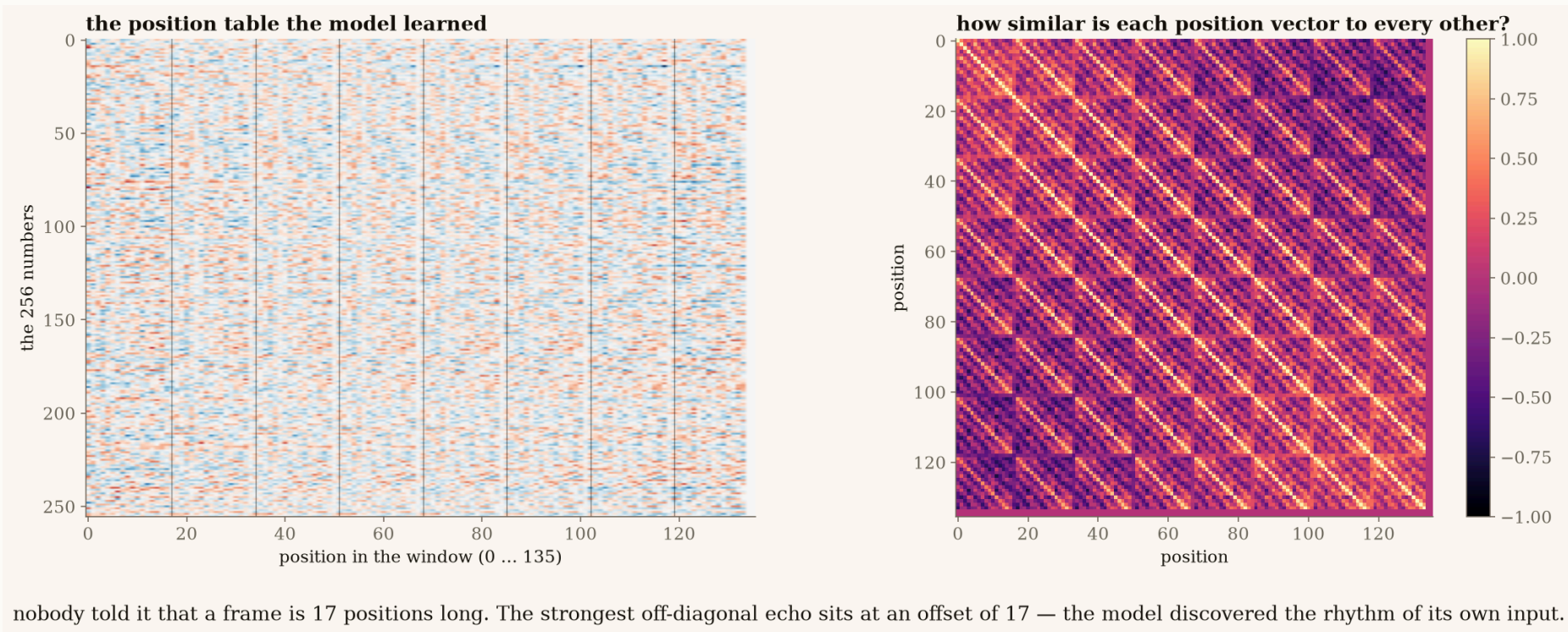
2 So position is added to the vector

A second learned table, one row per slot in the window, added straight onto the word embedding. Position 0 and position 135 get different rows, so the same word in two places becomes two different vectors.

3 Note what this costs

The table has exactly 136 rows. The model *cannot* be shown a longer window than that without learning new ones. The RSSM had no such limit — it is the first real price of keeping the past instead of summarising it.

Step 3 in the real model – the position table it learned



left: the actual learned table, one column per slot · right: how similar each position vector is to every other — and the strongest off-diagonal echo sits at an offset of exactly 17

Nobody told it that a frame is seventeen positions long

1 The position table starts as 136 rows of zeros

No sine waves, no hand-designed structure — just free parameters. Whatever pattern is in there was put there by the loss.

2 And it grew a period of 17

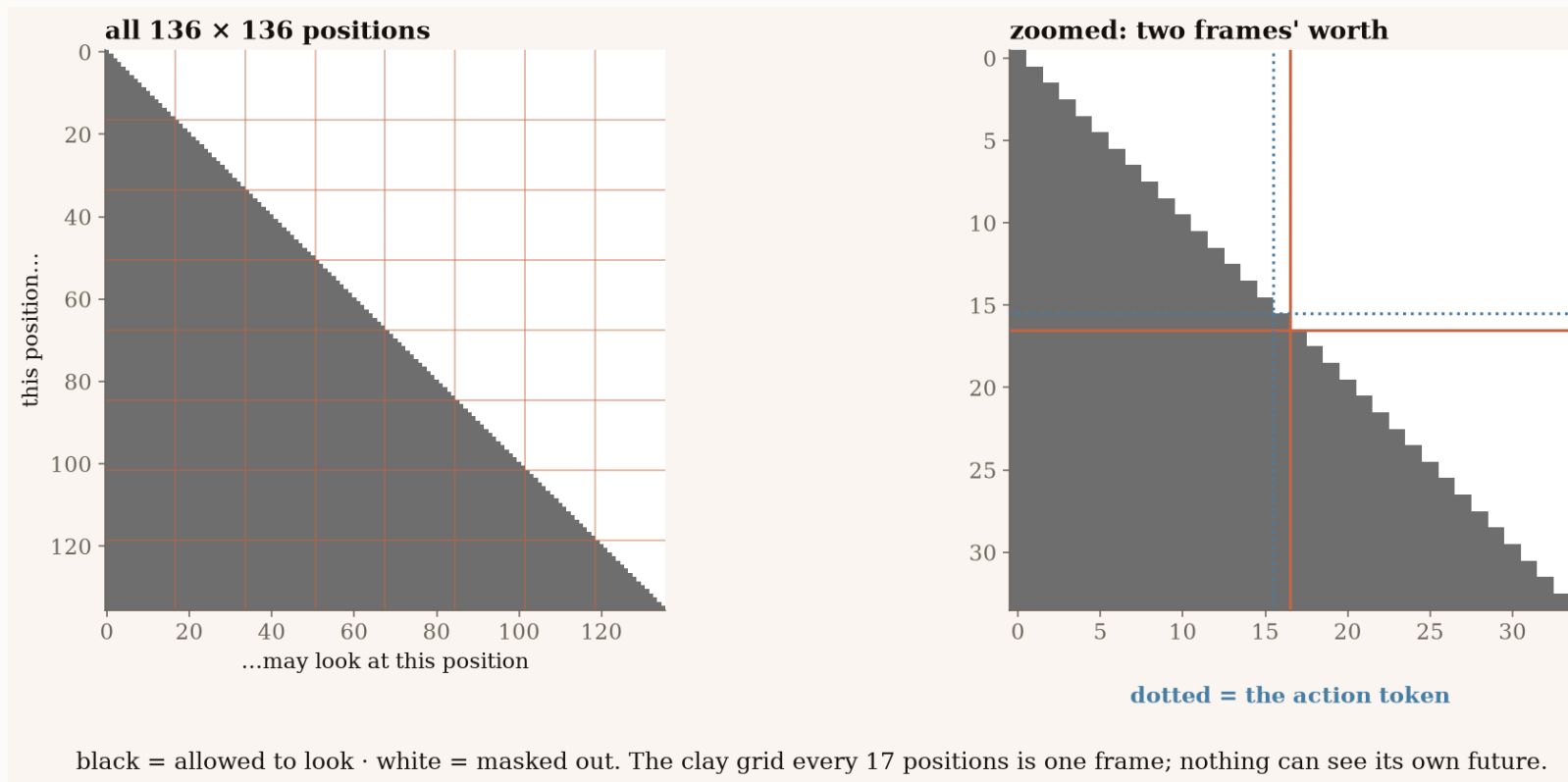
Positions 17 apart end up with the *most similar* vectors of any offset beyond ten. That is exactly one frame: 16 picture words plus one action. The model worked out the rhythm of its own input from nothing but next-token prediction.

3 Why it is useful to it

"I am the 3rd patch of a frame" is a genuinely helpful thing to know when you are predicting the 4th. The table gives every position a way to say which slot of which frame it is.

A caution worth keeping: this is one small model on one game. It is a nice demonstration that structure can be learned rather than imposed — not evidence that it always is.

Step 3 – the causal mask, and its seventeen-position heartbeat



black = allowed to look, white = masked out · the **orange** grid marks every 17th position, where one frame ends and the next begins · the **blue dotted** line marks the action token — the 17th slot of each step, which is the position whose output predicts the next frame's first word

This triangle is why transformers train fast

1 Without it, the model would cheat

Position 40 predicting position 41 while being allowed to look at position 41 learns nothing. The mask makes "predict the next one" honest.

2 With it, every position trains at once

One forward pass produces 136 predictions, each conditioned only on its own past. An RNN would need 136 *sequential* steps to do the same work.

3 Look again at Part 1's cost table

"Recurrent: strictly sequential. Attention: every position at once." That line was about this triangle. It is the single biggest reason transformers scaled and RNNs did not.

One asymmetry worth naming: this parallelism is a **training** property. When we *imagine*, tokens must still come one at a time — which is exactly the 16-sequential-decodes bill we will measure in Part 5.

"One pass gives 136 predictions? I thought we predicted one at a time."

Both are true, and they are different situations. This is the single most confusing thing about transformers, so it is worth being very explicit.

1 When TRAINING, we already know the answers

The whole 136-token sequence comes from a real episode, so the correct next token at every position is already on disk. The mask stops each position seeing its own future, so all 136 guesses are honest — and we can score every one of them in **one pass**. That is what "trains in parallel" means.

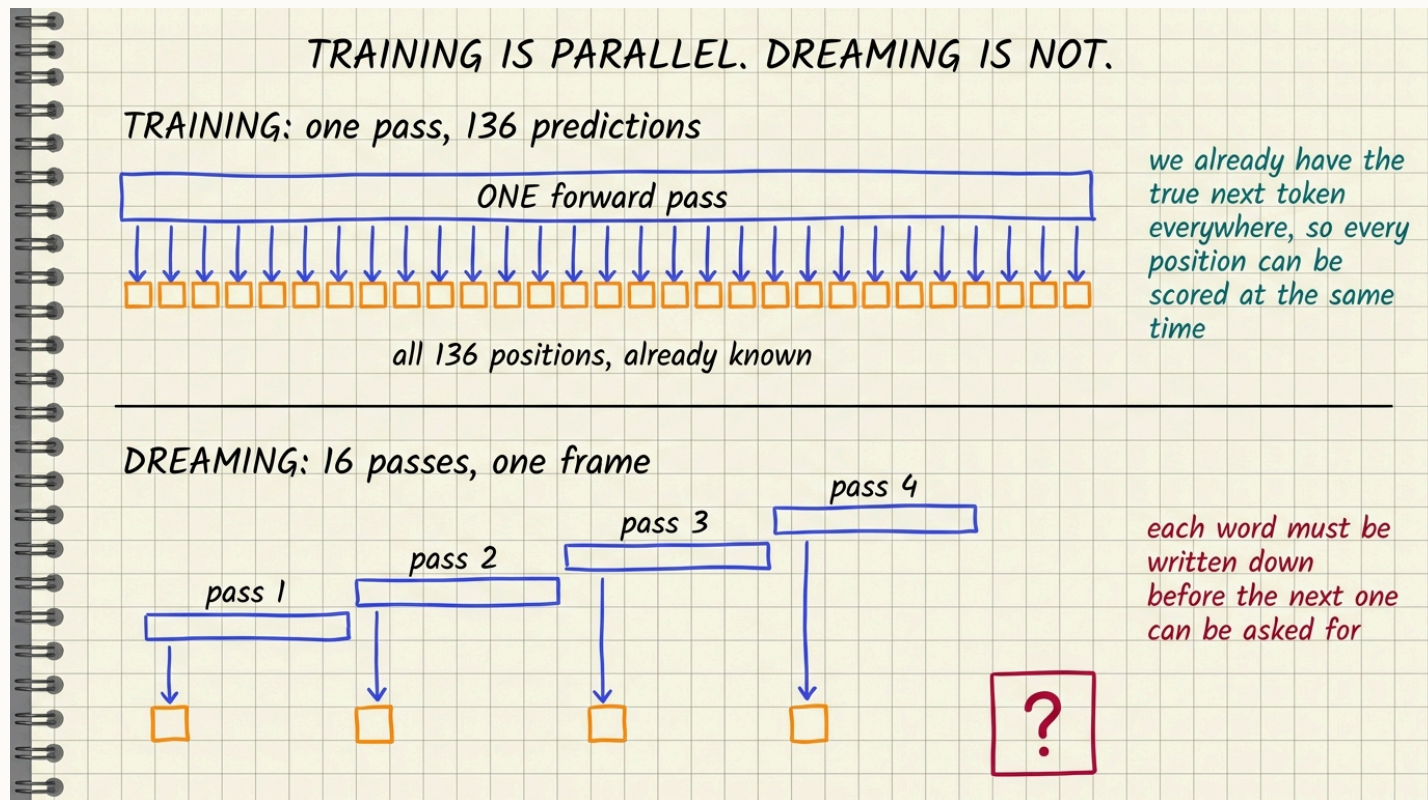
2 When IMAGINING, we know nothing

There is no future to feed in — we are inventing it. So we run the stack, sample *one* word, write it into the sequence, and run the whole stack again. **16 passes for one imagined frame.**

3 Same weights, same mask, completely different cost

Nothing about the network changes between the two. This asymmetry — cheap to train, expensive to dream — is the bill we measure at the end of Part 5.

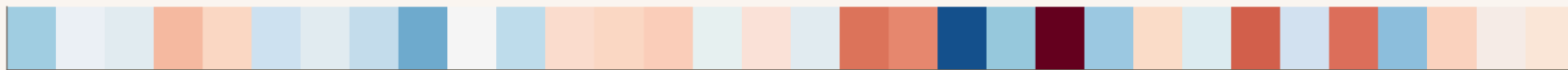
Training is parallel. Dreaming is not.



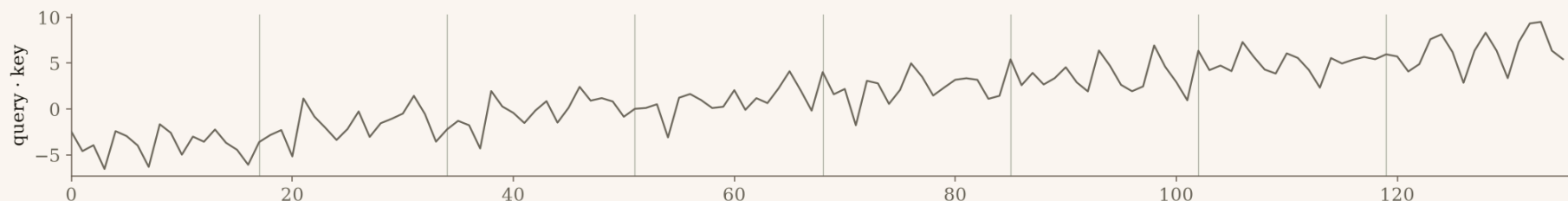
the same network, run two different ways · the top is why transformers scaled; the bottom is why imagining with one is slow

Step 4 – inside one head: query, key, value, with the real numbers

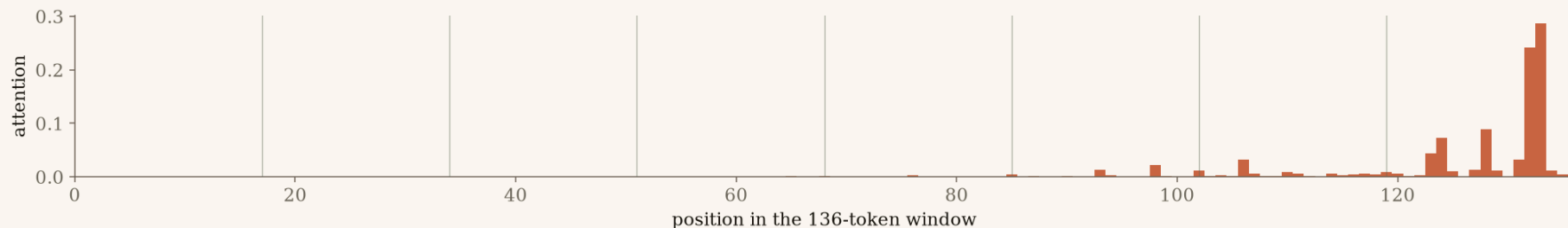
the QUERY built at position 135 (32 numbers) — "what am I looking for?"



...dotted against every KEY before it — one score per position



...softmax turns the scores into weights that sum to 1. The VALUES are then blended in exactly these proportions.



position in the 136-token window
layer 3, head 1, real weights — the top 81% of this head's attention sits on just 8 of the 136 positions

layer 3, head 1, traced from the trained model · a query is built, dotted against every key before it, and softmaxed into weights that sum to one

Three different questions asked of the same vector

Every position holds one 256-number vector. Three separate matrices turn it into three *different* vectors, each with a different job. Same source, three roles.

1 Query — "what am I looking for?"

Built from the position doing the asking. Here: the last action token, about to predict the next frame's first word.

2 Key — "what do I have on offer?"

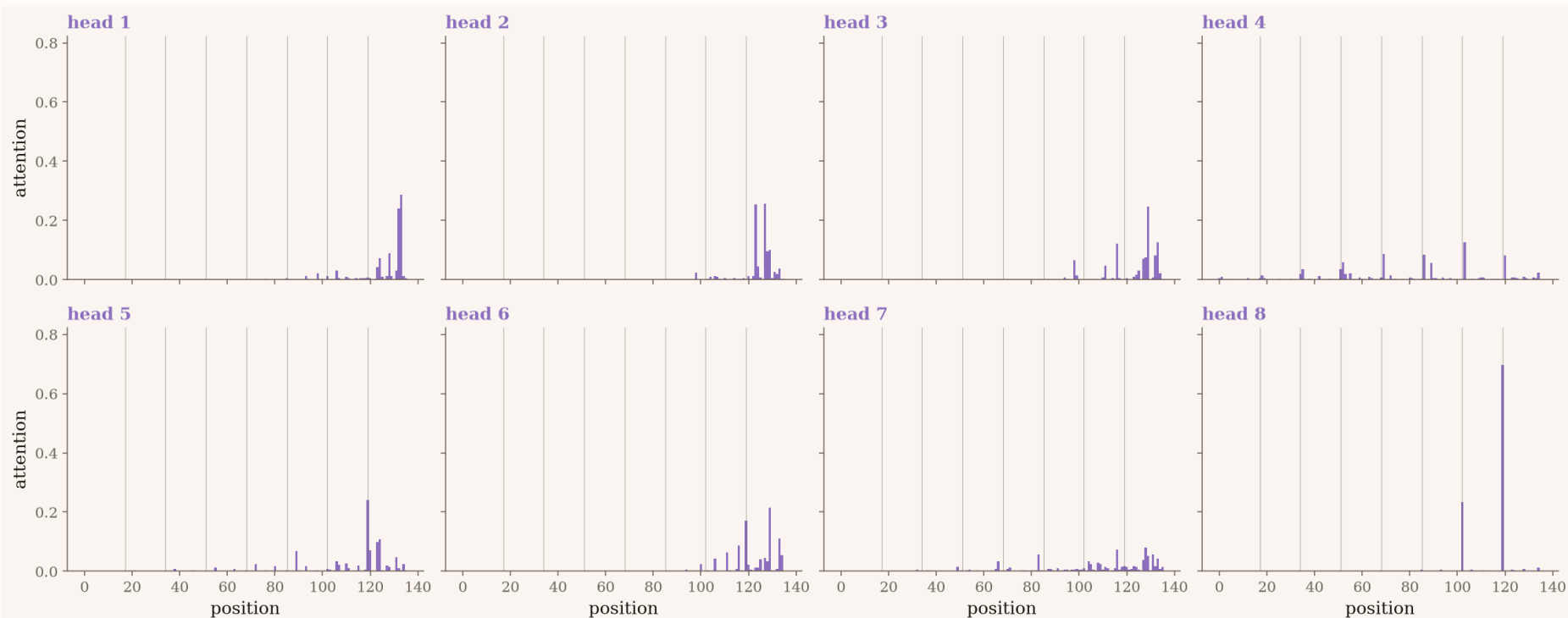
Built from every position that might be looked at. The dot product of a query with a key is the match score — big when they point the same way.

3 Value — "and here is what you get if you pick me"

A third vector, carrying the content actually handed over. The output is the values, blended in the softmaxed proportions.

The scores are divided by $\sqrt{32}$ before the softmax. Without it, the dot products grow with dimension, the softmax saturates, and gradients vanish — a one-line detail that decides whether the thing trains at all.

Step 5 – eight heads asking eight different questions



the same position, layer 3, all 8 heads · each head asks its own question and they are visibly different — their answers are concatenated and mixed by one more matrix

same position, layer 3, all eight heads · they are visibly not doing the same job — their outputs are concatenated and mixed by one more matrix

256 numbers split eight ways, not eight copies of the same thing

1 Each head gets 32 of the 256 dimensions

Not eight full-size attentions — one attention, carved into eight independent 32-dimensional subspaces. The cost is the same; the expressiveness is not.

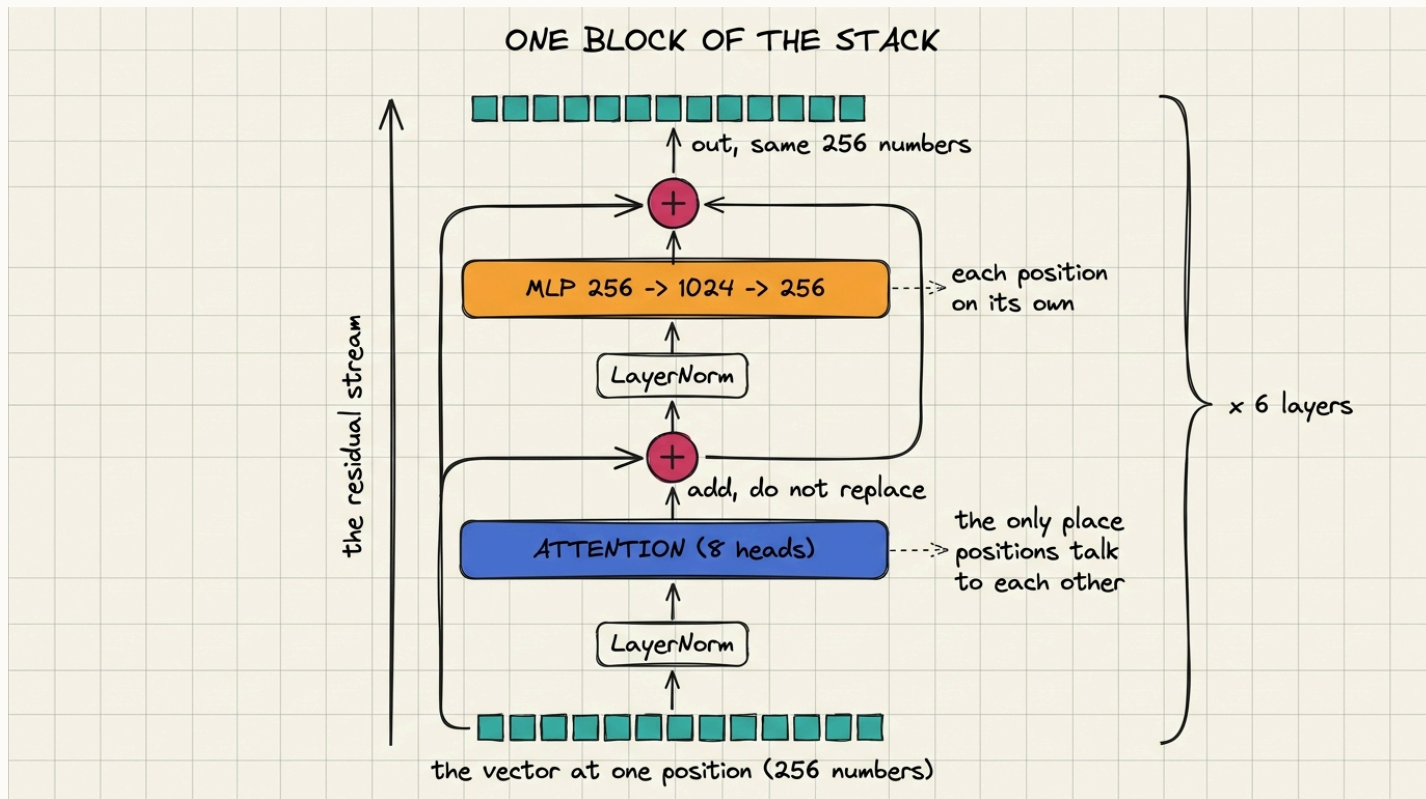
2 One head can only make one blend

Its softmax sums to one, so pointing hard at the patch above forces it to give up looking at the patch to the left. With eight heads you can do both at once.

3 Then they are stitched back together

Concatenate the eight 32-number results back into 256, and pass them through an output matrix that decides how to combine them. That is the complete attention sublayer.

Step 6 – the rest of a block: add, normalise, and a small MLP



attention is only half of a block · the other half is a two-layer MLP applied to each position on its own, and the two residual additions that keep the stream intact

What the MLP and the residual are actually for

1 Attention moves information; the MLP processes it

Attention is a weighted *average* — it can only ever gather. The MLP, $256 \rightarrow 1024 \rightarrow 256$ with a nonlinearity, is where the gathered information is turned into something new. Two thirds of the parameters live here.

2 The MLP sees one position at a time

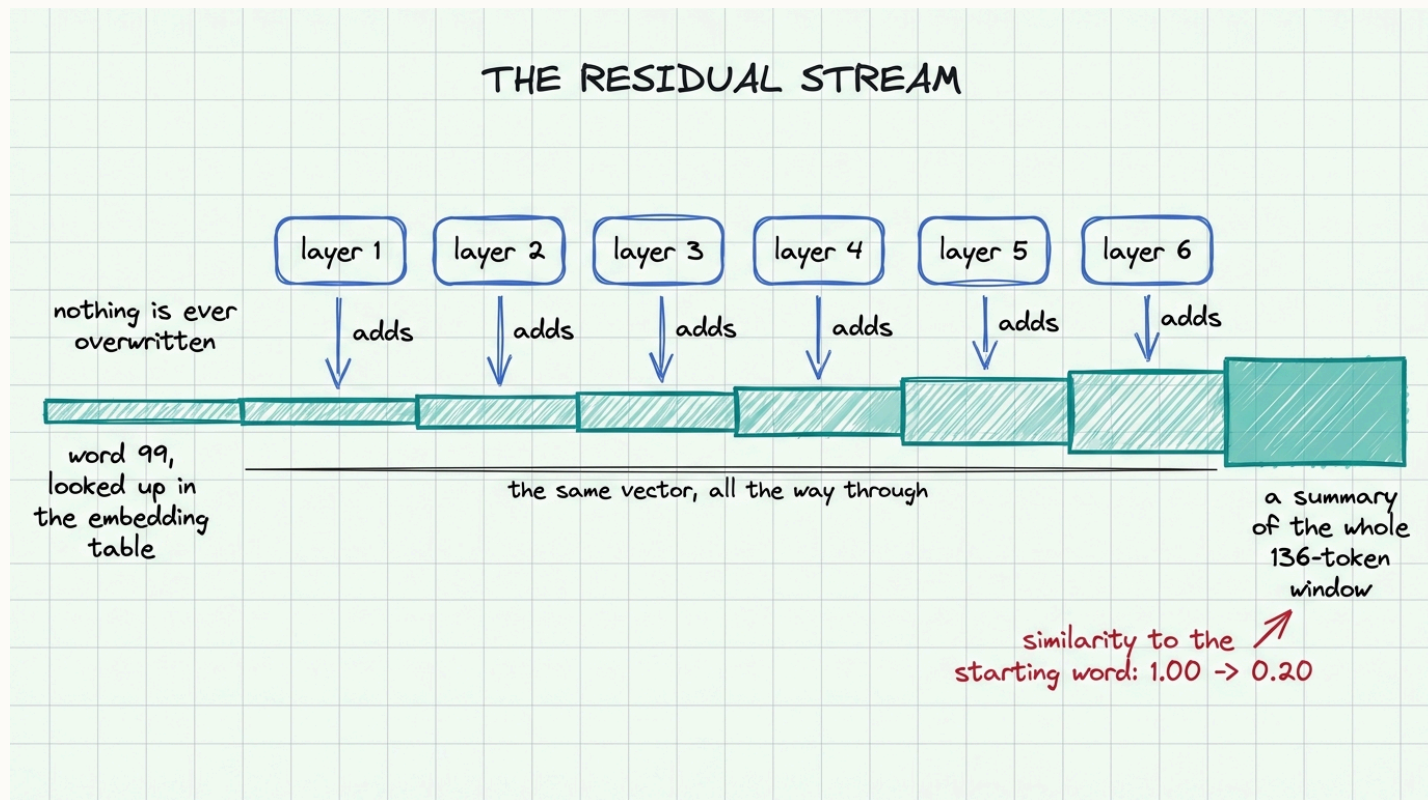
No mixing. Every position is processed by the same small network, independently. All communication between positions happens in attention and nowhere else.

3 The residual is why six layers can be stacked

Each sublayer *adds* to the running vector rather than replacing it. The stream is never overwritten, so a gradient can reach layer 1 directly — and each layer only has to learn a correction, not the whole answer.

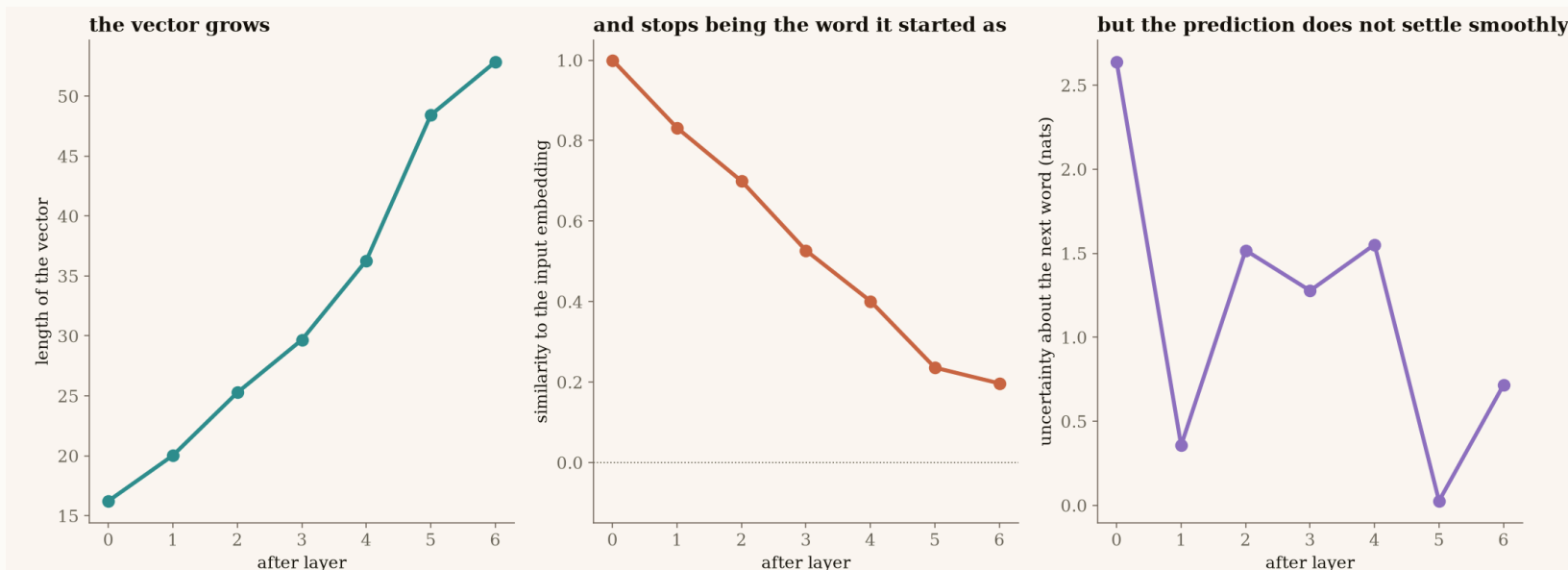
We normalise *before* each sublayer, not after (pre-norm). It makes deep stacks trainable without a learning-rate warm-up, and it is what almost everything modern does.

Step 7 – where the "context vector" actually comes from



the vector at one position is never replaced — every sublayer adds to it · what arrives at the top is the input embedding plus twelve corrections

And here is that happening, measured on our model



One position, traced through the stack. By the top of the network the vector has almost nothing left in common with the word that was fed in (cosine $1.00 \rightarrow 0.20$). It is no longer a word — it is a summary of the whole window.

one position, traced through all six layers · it always has 256 numbers; what changes is their *magnitude* ($16.2 \rightarrow 52.8$) and what they mean (similarity to the starting word $1.00 \rightarrow 0.20$)

What actually happens to the vector

This is the slide the whole part was built for. The thing people call "the context vector" is not produced by a special mechanism — it is **this stream, at the last position, after the last layer.**

1 It starts as one word and ends as a summary

Cosine similarity to its own input embedding falls from 1.00 to **0.20**. By the top of the stack there is almost nothing left of "word 99". It has been overwritten — by addition — with information pulled from the other 135 positions.

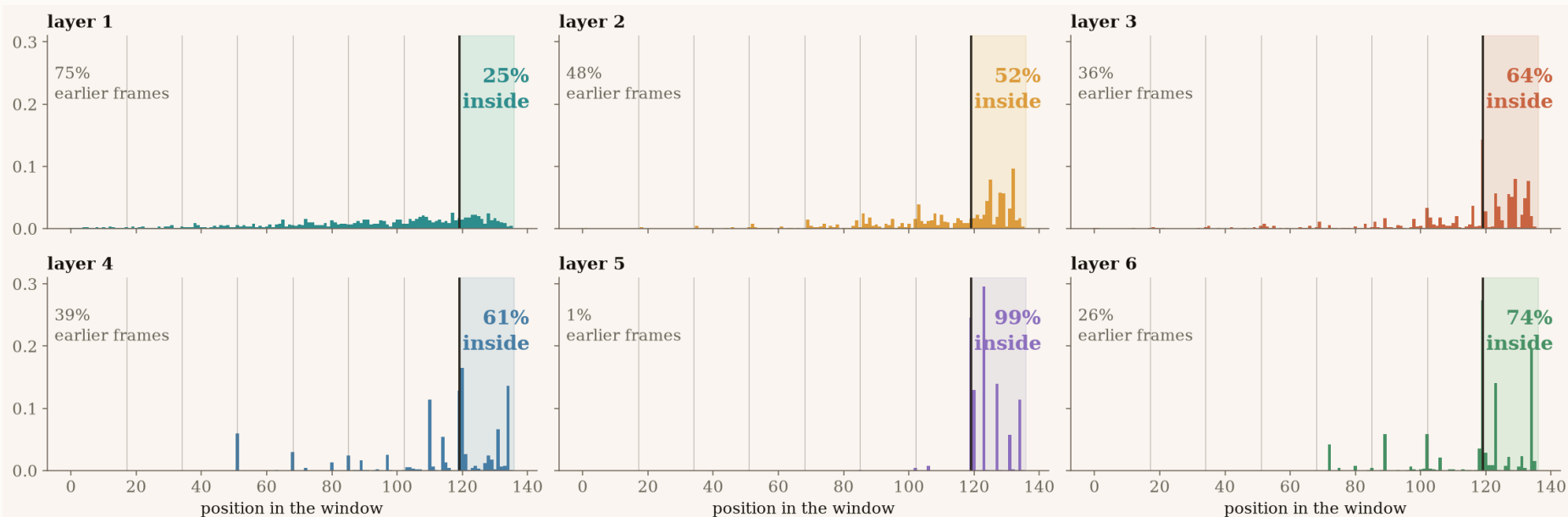
2 It gets *bigger*, not longer

Be careful with the word "length" here. The vector always has exactly **256 numbers** — that never changes. What grows is its **magnitude**: 16.2 → 52.8. Each layer adds more into the stream than it cancels, so the numbers themselves get larger. The vector accumulates rather than transforms.

3 And every position is doing this simultaneously

There is no single context vector. There are 136 streams running side by side, each summarising its own past. We only read the last one because that is the one being asked a question.

What each layer chooses to look at – measured, not assumed



attention from the sampling position, averaged over heads · the SHADED band to the right of the black line is the newest frame
share of attention INSIDE the newest frame, layer 1–6: 25% 52% 64% 61% 99% 74%

the shaded band in each panel is the newest frame, and the big number is the share of attention falling inside it · layer 1 → 6: 25% · 52% · 64% · 61% · 99% · 74% — early layers reach back across history, later ones settle onto the frame being finished

Nobody designed that split — it emerged from training

1 Layer 1 spends three quarters of its attention on the past

Only 25% inside the newest frame. It is gathering context — where the player was, which way things were moving.

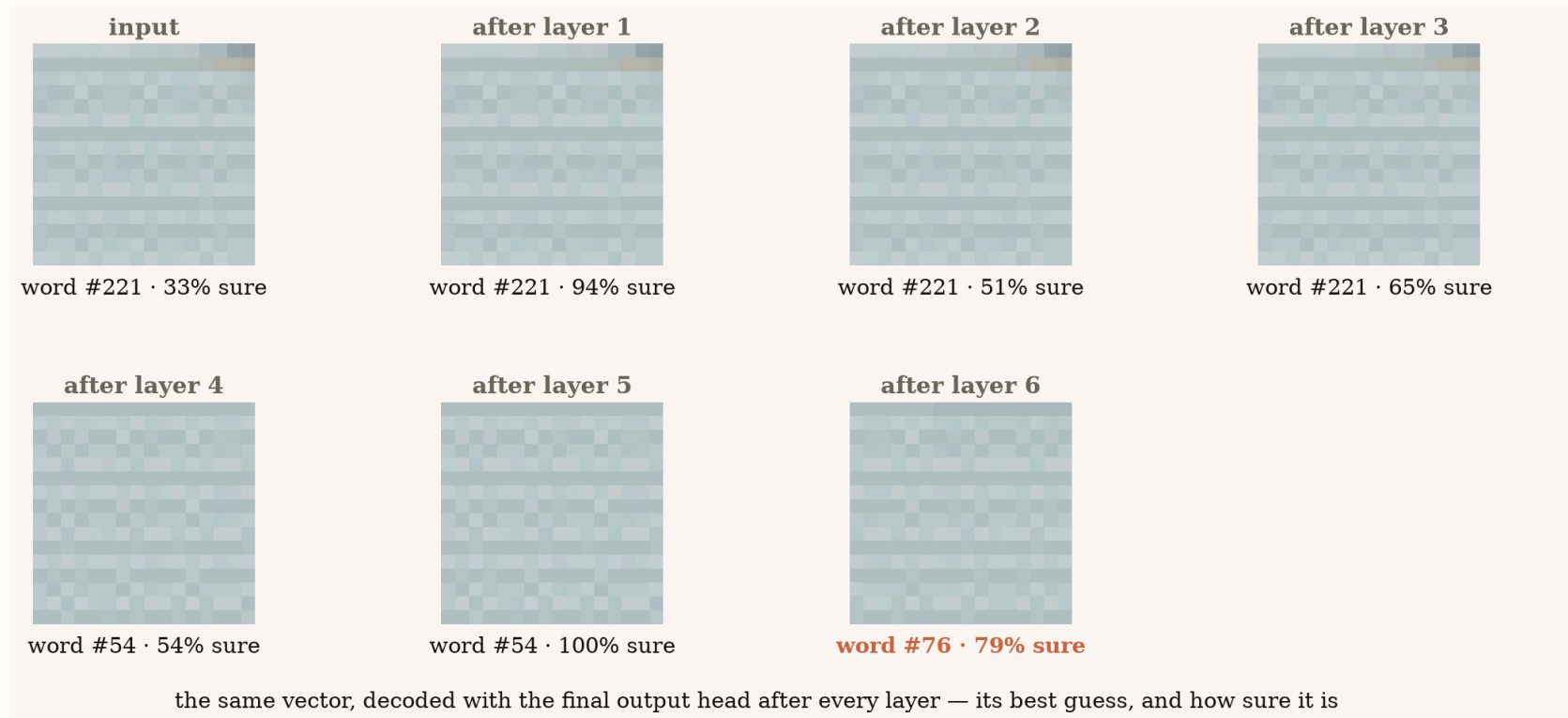
2 Layer 5 puts 99% of it on the current frame

By then the context has already been folded into the stream. The upper layers are doing local work: given everything I now know, what belongs in *this* patch?

3 Gather low, decide high

A pattern you will see again and again in transformers, and we did not build it in — the loss did. It is also a fair warning: this is one position, on one episode, in a small model. It is evidence, not a law.

Step 8 – asking the vector what it thinks, after every layer



the same vector, decoded with the *final* output head at every stage · its best guess is word 221 for four layers, flips to 54, and lands on **76** — the answer is assembled on the way up, not looked up

A trick worth knowing: decode the middle of the network

Because the residual stream is never replaced, an intermediate vector lives in the *same space* as the final one. So you can take the output head — trained only for the top — and point it at layer 3. Nothing stops you.

1 The guess changes twice

221 → 221 → 221 → 221 → 54 → 54 → **76**. The network does not know the answer early and refine it; it changes its mind, late.

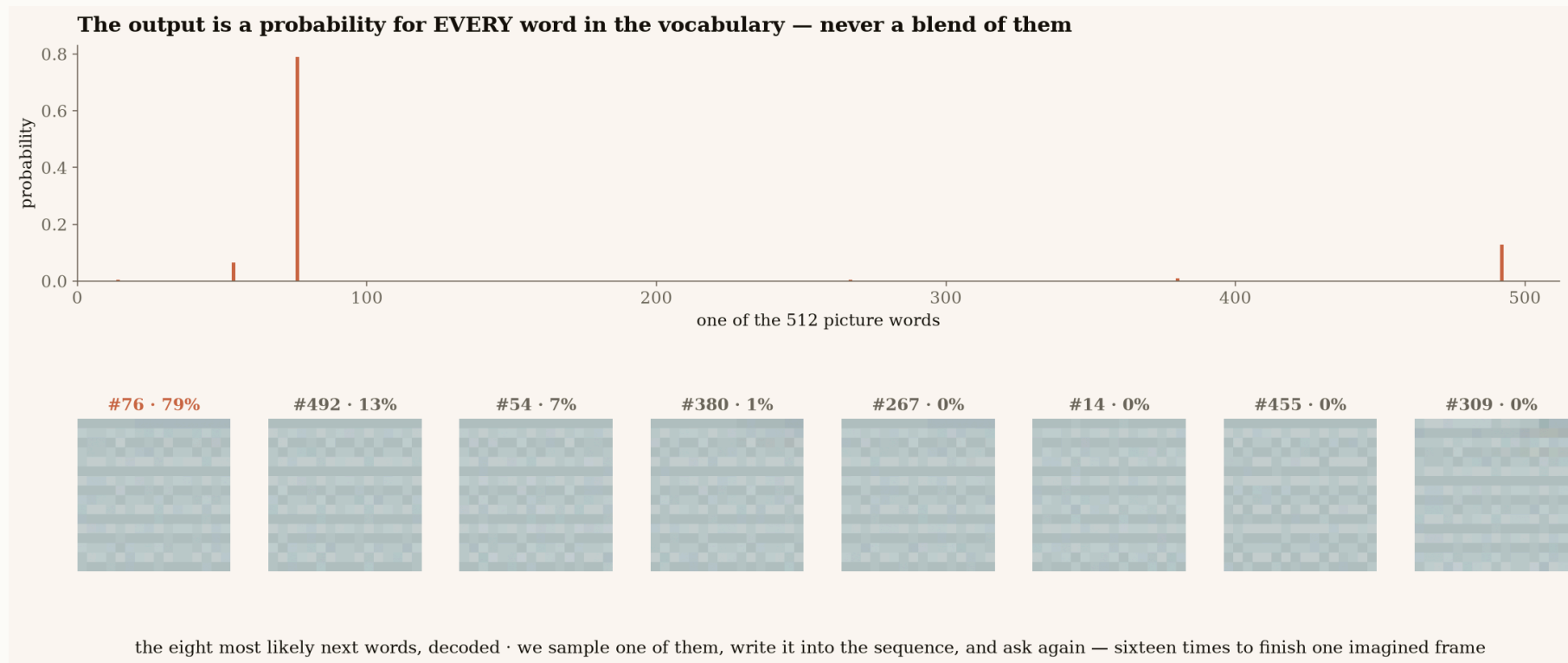
2 Confidence does not rise smoothly either

Uncertainty falls, spikes, and falls again. "Deeper = more certain" is a story we tell; the measurement does not support it.

3 And this only works because of the residual

In a network where each layer *replaced* its input, layer 3's vector would be in a private space and the final head would read noise. Interpretability here is a direct gift of the skip connection.

Step 9 – out of the top: a probability for every one of the 512 words



word **76** at 79%, word **492** at 13%, word **54** at 7% · three genuinely different continuations, each a real picture — this is Part 2's argument, in the actual output layer

Look at that distribution and remember Part 2

1 Three real candidates, not one blurred compromise

79 / 13 / 7. The model is genuinely unsure between three different-looking patches — and it says so, by naming all three. Each is a word that exists.

2 A Gaussian head could not have done this

It would have had to output one point. The point between three different patches decodes to the blur we diagnosed in Lecture 4. *This slide is why the whole lecture is called "a vector or a vocabulary".*

3 Then we sample, write it down, and go again

Draw one word, append it to the sequence, re-run the stack. Sixteen times for one imagined frame. That loop is the world model — and its bill is the last measurement in Part 5.

Nine steps, start to finish

	WHAT HAPPENS	SHAPE
1	frame → sixteen picture words, plus one action	136 integers
2	each integer looks up the transformer's own embedding	136×256
3	a learned position vector is added	136×256
4	$\times 6$ · normalise, then attend under the causal mask	$8 \text{ heads} \times 32$
5	$\times 6$ · add the result back into the stream	136×256
6	$\times 6$ · normalise, MLP $256 \rightarrow 1024 \rightarrow 256$, add back	136×256
7	a final normalise	136×256
8	one matrix to vocabulary size, then softmax	136×512
9	sample the last position · repeat $16\times$ for a frame	1 word

That is the entire model. **6.9 million parameters**, no recurrence, no hidden state carried between steps — and the "memory" is nothing more than the fact that the old tokens are still sitting in the window.

Explanation from Rajat

— WHITEBOARD —



PART 5 OF 5

Head to Head

By the end of this part you should be able to say: what each latent space actually looks like from the inside, which model dreams more sharply, how far back attention really needs to look — and what each design costs.

COINRUN · BOTH MODELS · MEASURED

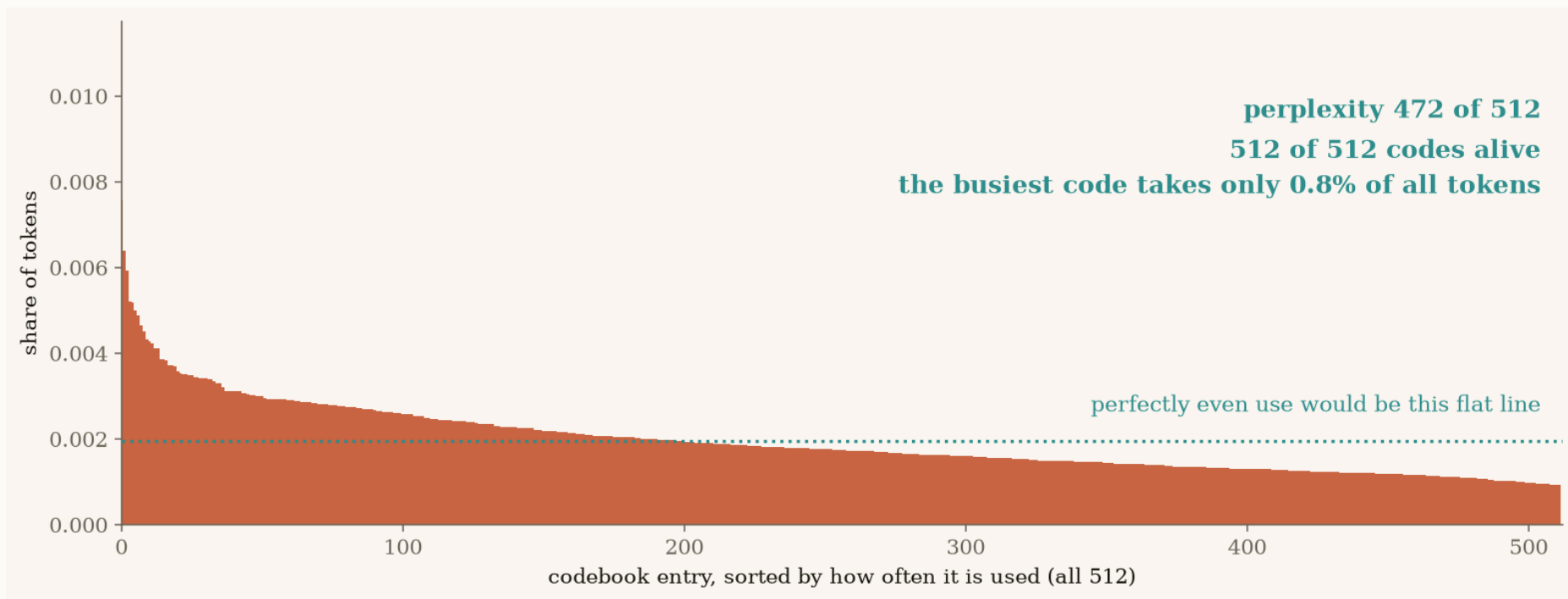
One game, two world models, everything else held fixed

We generated 600 CoinRun episodes — 256,151 frames — with a random policy, and trained two world models on exactly the same data.

	MODEL A • RSSM	MODEL B • IRIS-MINI
state	32 continuous numbers	16 words from a 512-word codebook
dynamics	GRU (Lecture 4's design)	causal transformer
latent kind	Gaussian	categorical
parameters	5,364,035	6,882,499 (tokenizer + transformer)

On fairness: the two are matched on *training wall-clock* and land within 28% on parameters, and 40 episodes were held out that neither model ever saw. It is still not a perfectly controlled comparison — IRIS changes two things at once — and we say so where it matters rather than burying it.

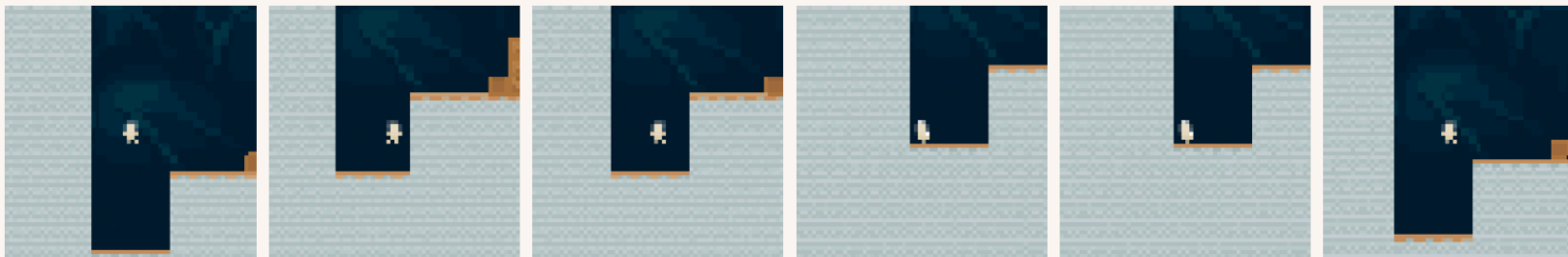
First: is the model actually using its dictionary?



codebook collapse is the classic silent failure · perplexity 472 of 512 · 512 of 512 codes alive — this dictionary is genuinely in use

Is sixteen numbers enough to hold a frame?

the frame



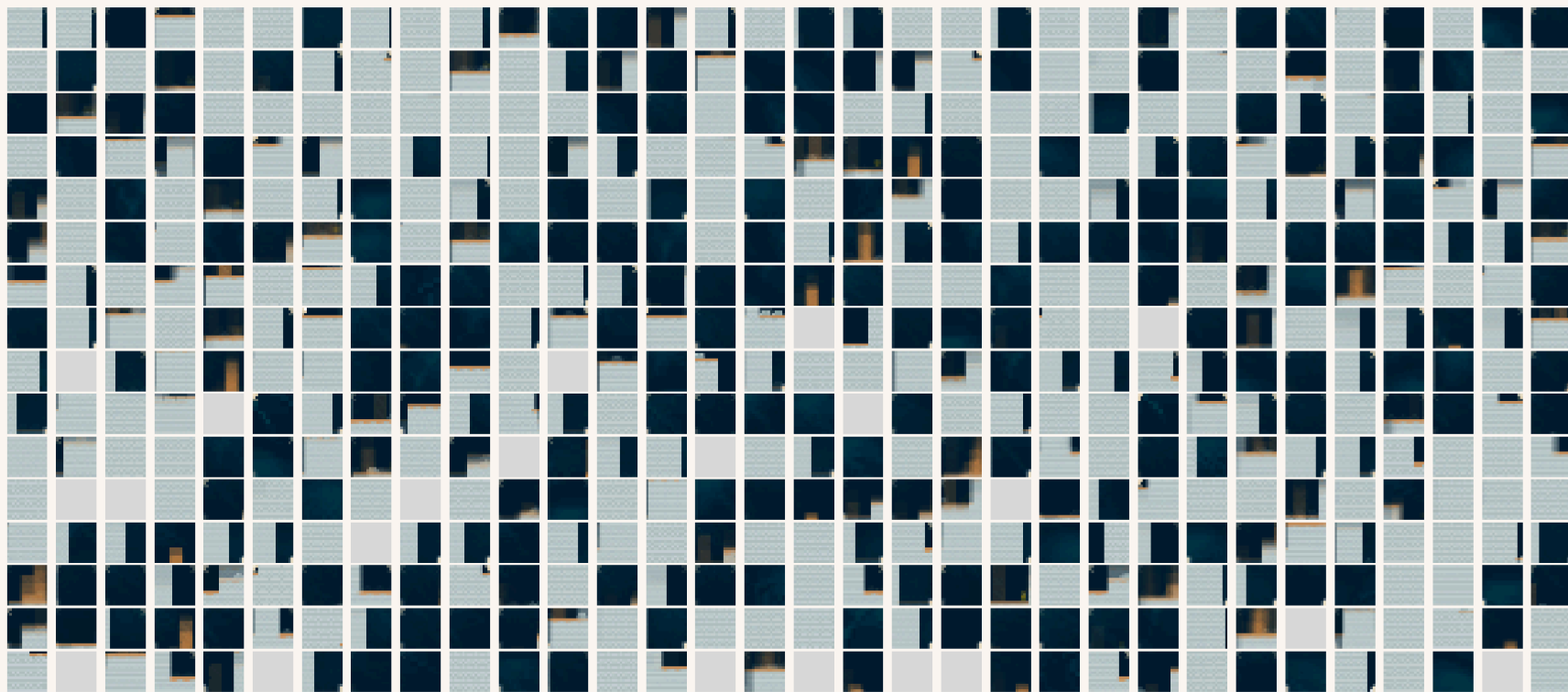
rebuilt from
16 numbers



pixel MSE 0.00081 on episodes the tokenizer never saw · 12,288 numbers in, 16 out, and the game is still legible

held-out episodes the tokenizer never saw · pixel MSE 0.00081 · 12,288 numbers in, 16 whole numbers out, and the game is still legible

The whole vocabulary: all 512 visual words this model invented for CoinRun



every square is one *word*, shown as the average of the real 16×16 image patches assigned to it · most-used first · nobody labelled any of these — the model invented the whole dictionary from raw frames

Two different things, and it matters which is which

Fair question: is the model *learning those pictures*? No. It learns 512 **vectors**. The pictures are how we made those vectors visible — and the honest way to do it is not obvious.

1 What the model actually learns

512 vectors of 64 numbers each, nudged during training toward the encoder outputs assigned to them (an exponential moving average). That is the whole codebook. No image is stored anywhere.

2 How we drew it

We ran **34,560 held-out patches** through the tokenizer, noted which word each 16×16 patch was assigned to, and **averaged the real patches per word**. So each square answers: "what does this word look like in the data?" 491 of the 512 appeared in that sample.

3 The way we tried first was wrong, and worth mentioning

We decoded each codebook vector on its own. That hands the decoder a *constant* field, and it dutifully paints a flat grey wash — real model output that shows nothing. A word only has a look because of the company it keeps.

And this is what "discrete latent" actually means

1

These are the model's only building blocks

Every frame it will ever describe, dream or predict is assembled from sixteen of these squares. There is nothing else in its world.

2

You can print the whole state space

That is Part 2's "enumerable" made completely literal. Try doing this for a 32-dimensional Gaussian — you cannot; there is no list.

3

And the model learned them unsupervised

From nothing but 256,000 frames and the demand to rebuild them. The structure of the game fell out of the compression.

The same picture, described two completely different ways

one CoinRun frame



IRIS · 16 words

336	351	132	30
375	66	273	339
453	443	276	69
353	45	115	52

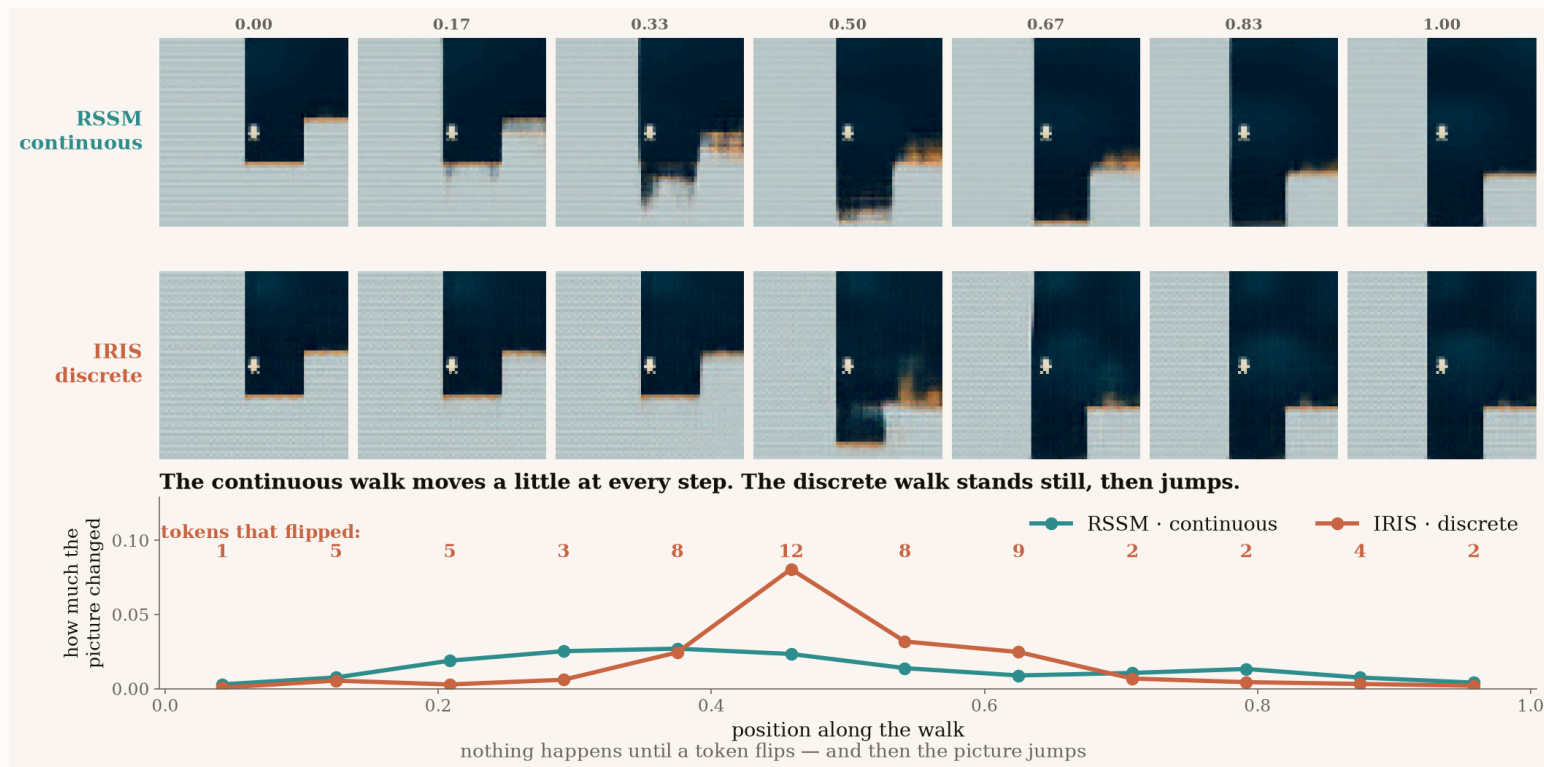
RSSM · 32 continuous numbers

+1.72 -2.16 -8.25 +2.42 +2.11 -1.69 -8.32 +6.51
-4.87 +1.61 -10.23 +10.09 -0.17 +1.42 +6.21 -7.14
+8.23 -2.07 -13.17 +5.41 +10.58 +1.10 -2.27 +0.04
-2.99 +3.23 -9.69 +0.11 +3.52 -0.87 -8.35 +5.33

*you can read the tokens as words in a dictionary.
you cannot enumerate the space these numbers live in.*

sixteen whole numbers you could read aloud · or thirty-two real numbers you could not · both rebuild the same frame

Walking between two frames, inside each latent space



the continuous space passes through states that never occurred in the game · the discrete space can only ever land on real words

The newspaper figure, now in pixels

This is the same experiment we ran on `baseball` and `hockey` in Part 2 — except now the "words" are visual, and the model invented them itself.

1 The top row morphs

Every intermediate frame is a genuine point in the RSSM's space — and most of them are scenes CoinRun could never produce. Smooth, and not always meaningful.

2 The bottom row jumps

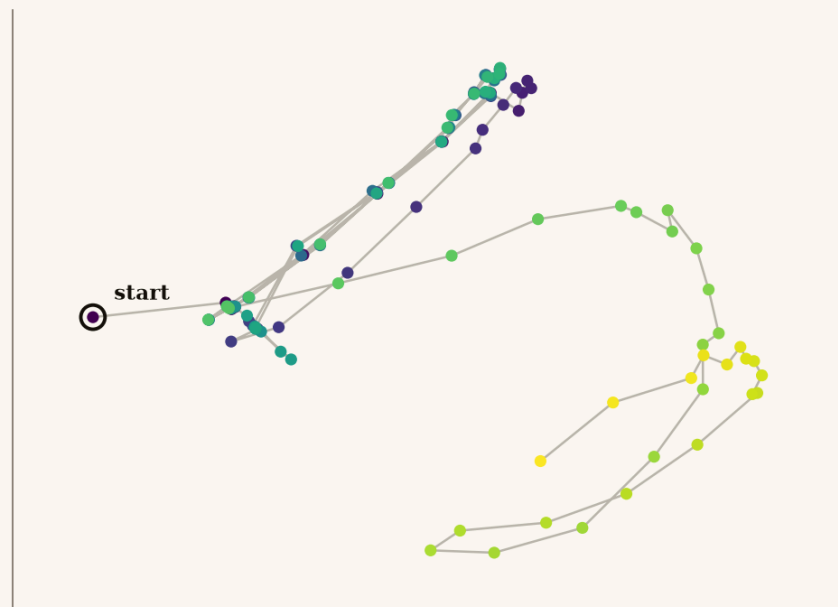
It snaps from one combination of real words to another, because there is nothing in between to land on. Discrete, and always assembled from real parts.

3 Neither is "correct"

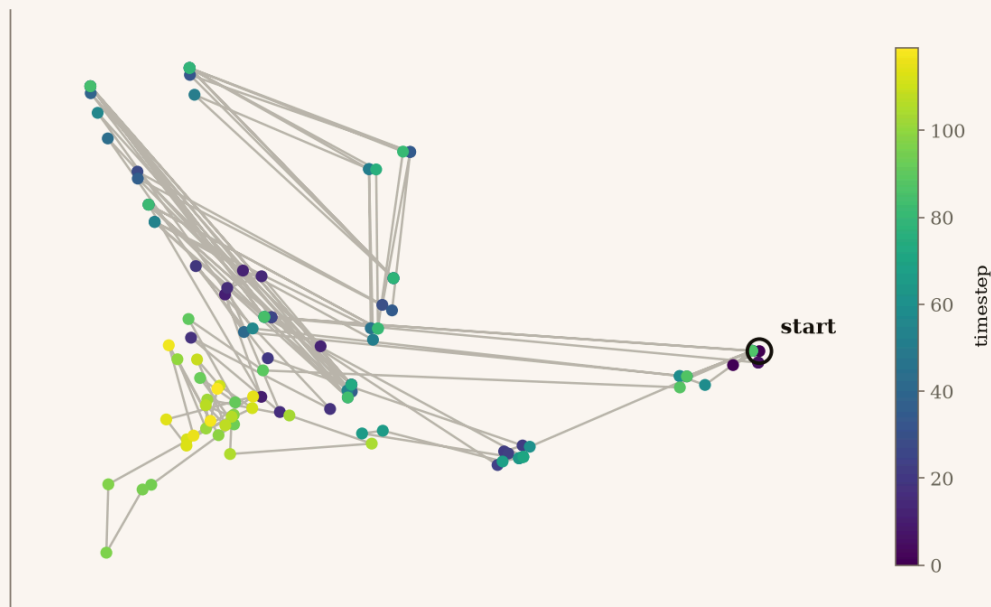
They are the two behaviours from Part 2's table, drawn with the same model that will do the dreaming. *This is the whole lecture in one figure.*

120 consecutive steps of one held-out episode, drawn in both latent spaces

RSSM · continuous latent



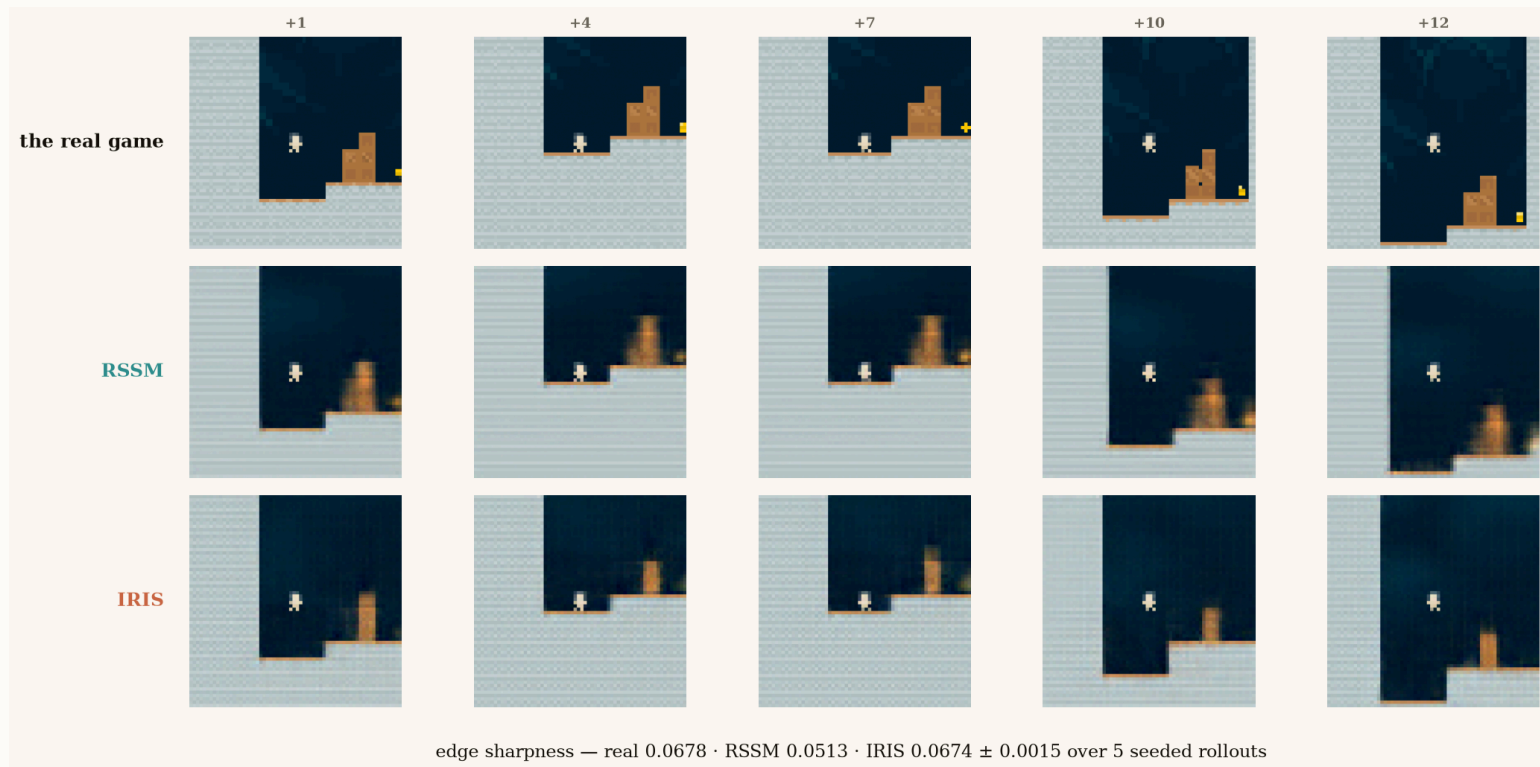
IRIS · bag of visual words



left: short, even steps along a smooth path · right: the state sits still, then jumps to another configuration

colour runs from the start of the segment to its end

Twelve imagined steps, same context, same actions



top: what really happened · middle: the RSSM's imagination · bottom: IRIS sampling tokens · edge sharpness 0.068 / 0.051 / 0.067 (IRIS averaged over 5 seeded rollouts, sd 0.002)

Did discreteness fix the blur?

1

What we predicted in Part 2

A categorical model cannot average two futures — every frame it paints must be assembled from real words. So its dreams should be *sharper*, even when they are wrong.

2

What we measured

Edge sharpness — real footage **0.068**, RSSM **0.051**, IRIS **0.067** ± 0.002 over 5 seeded rollouts. The token model's dreams are 1.31 $\times$ sharper than the RSSM's. And note where each one lands: the RSSM is *below* the real game — too smooth, which is what averaging futures looks like; IRIS is essentially *level with* the real game.

3

The distinction worth holding on to

Sharp is not the same as correct. A token model can produce a crisp frame showing something that never happens; a Gaussian model produces a soft frame that is roughly right. Which failure you prefer depends entirely on what you are going to do with the dream.

Why the demo feels broken for the first two presses

Worth a slide because it is the first thing a student will notice, and the wrong conclusion is right there waiting: *"the model is ignoring my action."* Press *left* instead of *jump* and the next imagined frame is almost unchanged — in **both** models. It is not ignoring you.

STEPS AFTER THE CHOICE	1	2	3	6
of 16 tokens, how many differ between the three actions	2.0	7.0	12.7	14.3
pixel difference between the futures — RSSM / IRIS	.004	.009	.039	.040
	.004	.012	.045	.050

1 This is a resolution limit, not a modelling failure

Each of our 16 tokens covers a 16×16 patch, and one frame of player movement is a few pixels. Often that does not change which word wins.

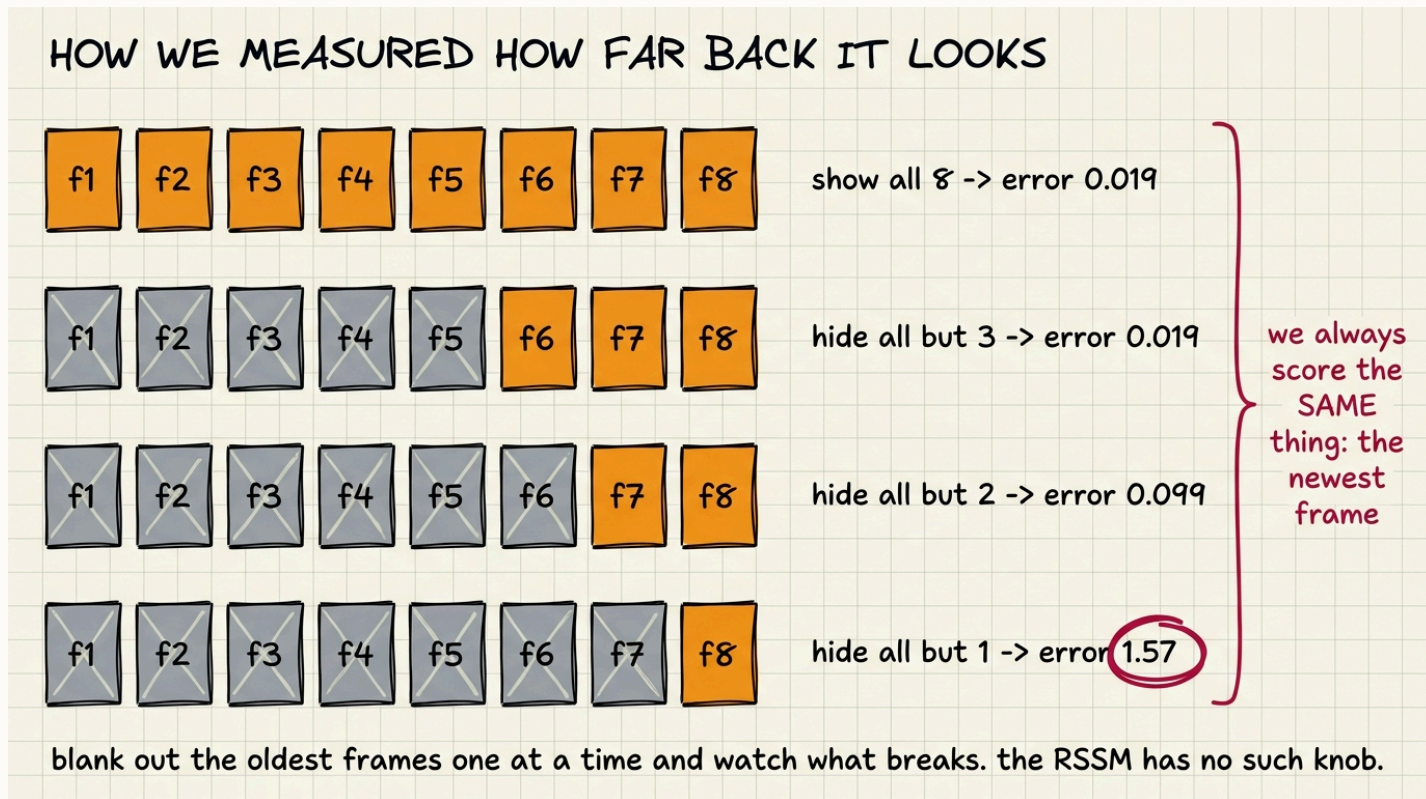
2 And it happens identically in both models

The continuous model shows the same delay — so this is about the world and the resolution, not about discreteness.

3 Consistent with what the transformer attends to

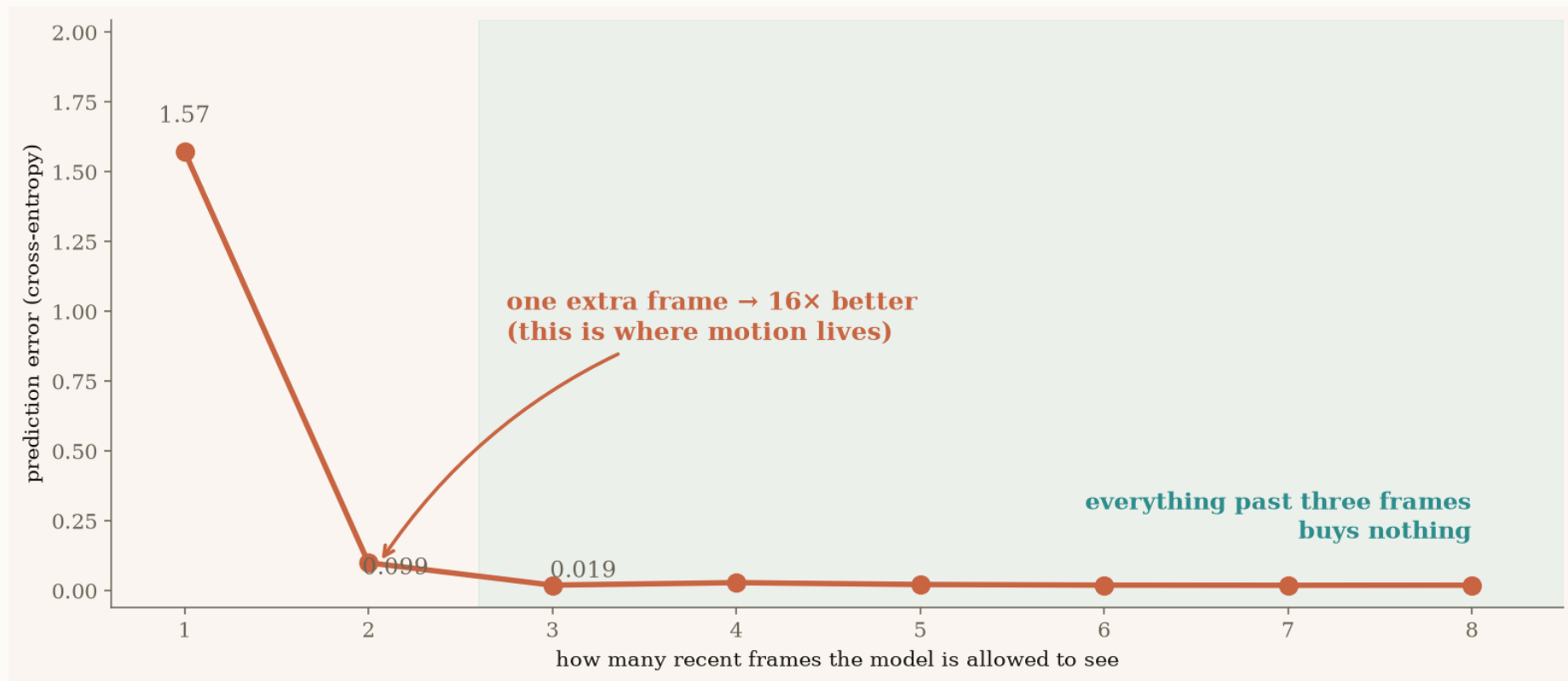
Only **0.1%–5.5%** of attention lands on action tokens. Actions are a slow, accumulating influence — not a switch.

How we measured how far back it looks



blank the oldest frames out, one at a time, and re-measure · we always score the *same* thing — the sixteen tokens of the newest frame — so the only variable is how much past the model is allowed

How far back does the transformer actually need to look?



one visible frame gives cross-entropy 1.57; two gives 0.099; three gives 0.019 and it is flat from there — on CoinRun the useful past is about 3 frames deep

We can ask this model a question the RSSM cannot answer

Blank the oldest frames, one at a time, and re-measure. Because the past is kept explicitly, we can literally *take it away* and watch what breaks.

1 The second frame is worth a factor of 16×

With one frame visible the model is badly lost. Give it one more and the error collapses. That second frame is where *motion* lives — you cannot see which way the player is running from a single still.

2 After about 3 frames, more history buys nothing

The curve is flat from there. On CoinRun, the useful past is shallow — which is an honest and slightly deflating result, and exactly why we measured it instead of assuming.

3 The RSSM has no equivalent knob

You cannot ask it to look one frame further back, because there is no "back". Whatever survived, survived. The transformer's memory is *inspectable* in a way the belt never was.

A fair caveat: a random policy on procedurally generated levels does not create many long-range dependencies. On a task that *needs* memory — the gripper hiding the cube in Lecture 4 — this curve would look very different.

What "imagined frames per second" actually counts

Before the chart: one **imagined frame** means one complete step of the future produced with the game switched off. We time how many of those each model can make per second, batch size 1, on the same A10G GPU. Nothing else is included — no rendering, no environment.

1 For the RSSM, one frame is one GRU step

Update **h**, draw **s** from the prior, and that is a complete future state. One small matrix multiply. (We do not even decode the pixels — an agent planning in latent space never needs to.)

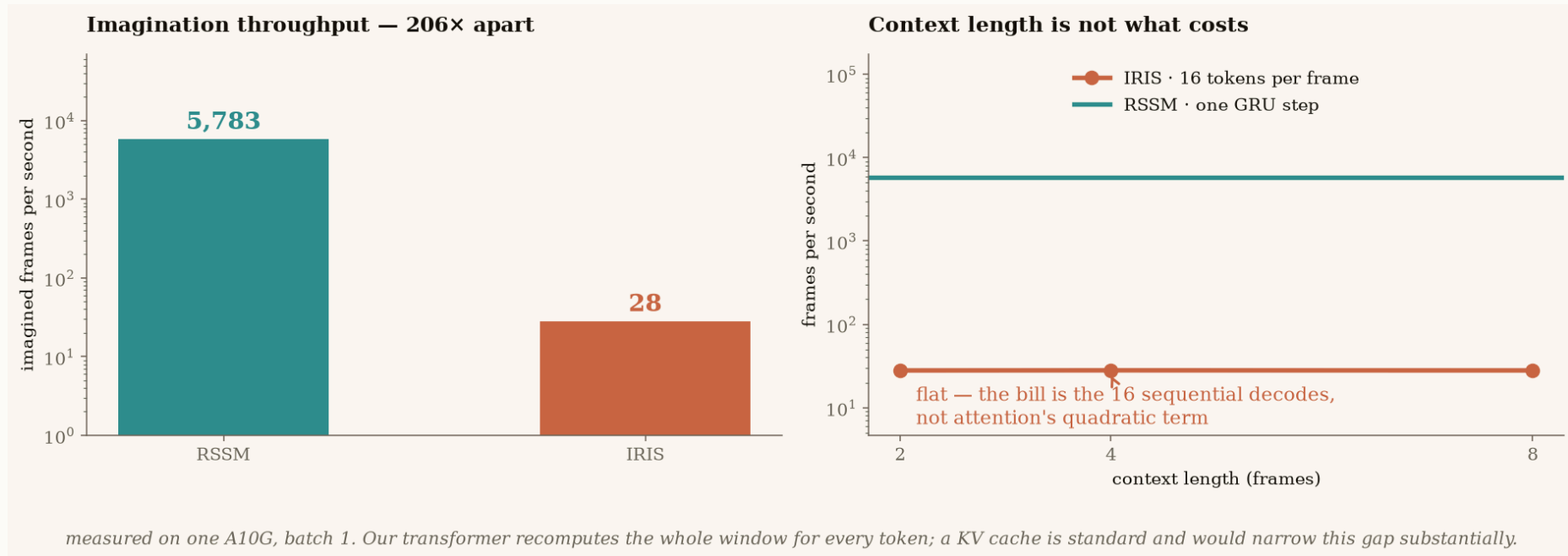
2 For IRIS, one frame is sixteen transformer passes

Sixteen words, and each one must be sampled and written down before the next can be asked for. Six layers, 136 positions, sixteen times over.

3 So the comparison is "one GRU step" against "sixteen forward passes"

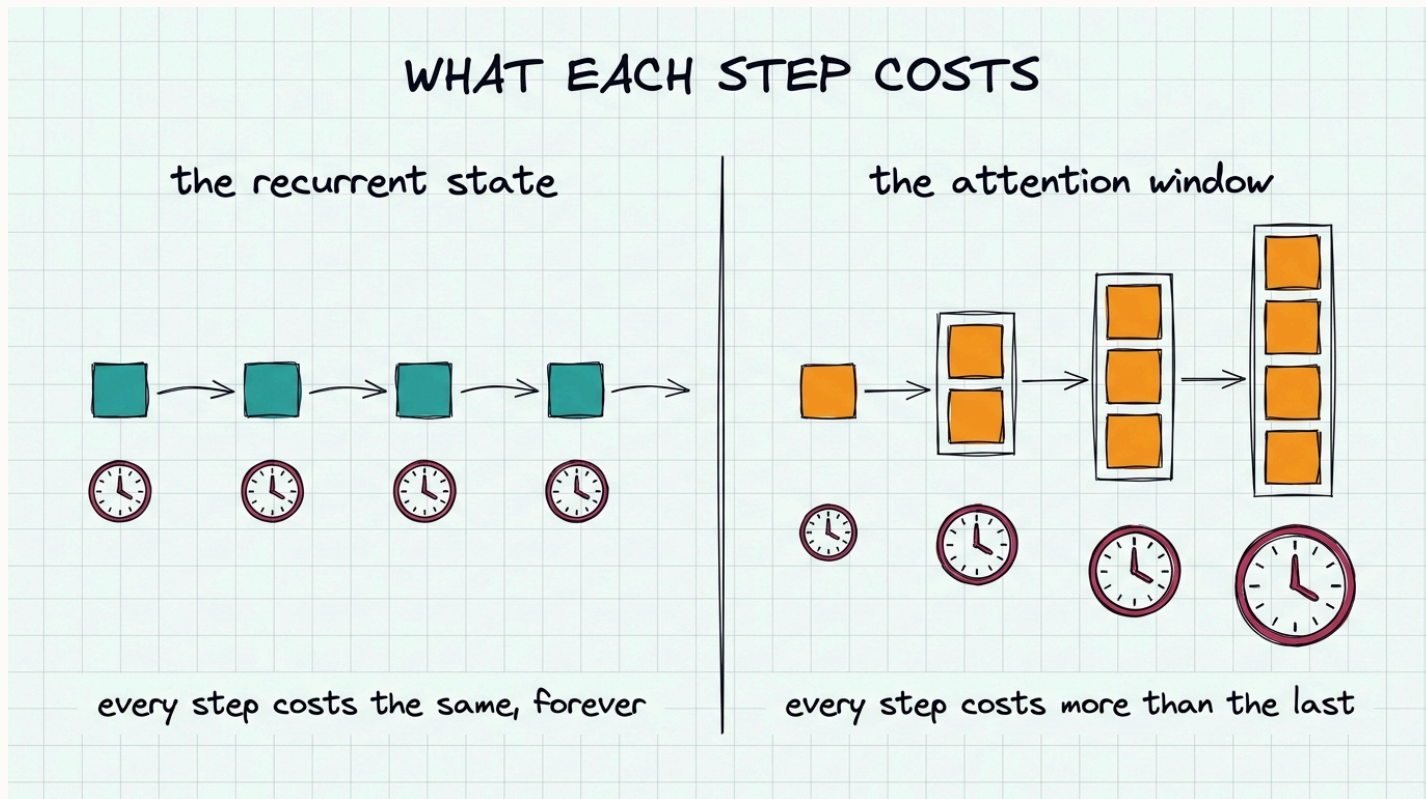
Stated that way the gap is not surprising at all. What *is* worth knowing is where the gap comes from — and it is not the place everyone assumes.

What each design costs to imagine with



5,783 imagined frames per second for the RSSM against 28 for IRIS — a factor of 206 · right: at this size the transformer's throughput is flat — 27.9 / 28.0 / 27.9 fps at 2, 4 and 8 frames — so the gap is the sixteen sequential decodes, not attention's quadratic term

Constant, or growing



the asymptotic picture, and the reason the recurrent design has not gone away · careful: this is cost against *how big you choose the window to be* — inside one rollout our window is fixed at 8 frames and never grows

Where the $200\times$ actually comes from – and it is not what you would guess

The textbook complaint about attention is that it grows with context length. On our benchmark, that is **not** what we measured – throughput was flat across contexts of 2, 4 and 8 frames.

1 The real cost is sixteen sequential decodes

Every imagined frame needs sixteen tokens, drawn *one after another*. Sixteen round trips through a six-layer transformer, versus one small GRU step. That is the gap.

2 The quadratic term is real, just not yet

Careful with what is fixed and what is not. In our model, a frame is always 16 words and the window is always 8 frames – **136 positions, and it never grows during a rollout**. The quadratic story is about choosing a *bigger window* in the first place, and at 136 the sequence is far too short for it to bite. The previous slide is right about the asymptotics and wrong about our numbers.

3 And there is real headroom we did not use

Our implementation recomputes the whole window for every token. A KV cache is standard and would narrow the gap substantially. We report the number we measured, with the caveat attached.

Nobody won — and that is why all four corners are still occupied

1 Discreteness bought honest forks

A model that can say "either this or that" instead of painting the average. Part 2 predicted it; here it is in pixels.

2 Attention bought perfect recall

No 24-word cliff. The past is kept, and the model chooses what to use.

3 The RSSM kept something neither of them has

Imagination that costs the same at step 1,000 as at step 1. When you want to imagine *thousands* of futures — which is exactly what an agent that plans will want — that constant cost stops being a detail and becomes the whole argument.

That tension is live in 2026. It is why Dreamer kept the recurrent state, why IRIS did not, and why our own CoinRun model sits in a fourth corner that takes attention *and* continuous latents.

Four things to keep

1 A running summary has a cliff, not a slope

Perfect at a 12-word gap, at chance by 24. Fixed-size memory does not degrade gracefully — it is overwritten.

2 Continuous and discrete are good at opposite things

One interpolates and averages. The other enumerates and chooses. Pick by asking whether your world varies smoothly or makes decisions.

3 A vocabulary can be learned, not given

512 visual words invented from raw frames, with nobody labelling anything. Always measure whether the model is really using them.

4 Sharp is not the same as correct

The most important sentence in this lecture. Choose the failure mode that suits what you will do with the prediction.

Next: do we need pixels at all?

Both models today spent most of their capacity on one job we never asked for: **repainting the background**. CoinRun's sky does not move, and neither model gets credit for knowing that — they just keep drawing it.

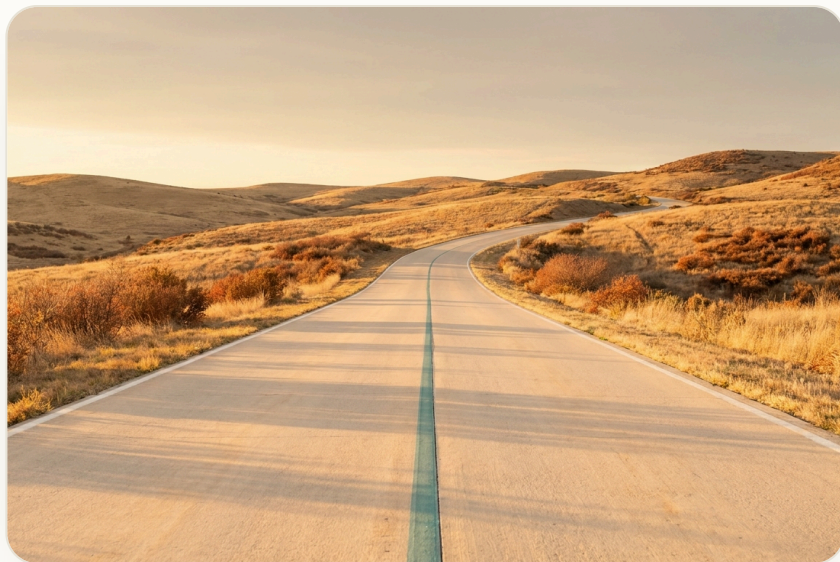
● What if the model never reconstructed a frame?

Predict in *representation* space instead — never paint a pixel. That is the JEPA idea, and DINO-WM builds a world model on exactly that bet.

● And the obvious cost

You can no longer *watch* the dream. We will have to find another way to know whether the model is right — which turns out to be the interesting part.

●●● Lecture 6 · Do we need pixels at all?



We taught it words. Next we ask whether it needs pictures.